POLITECNICO
DI TORINO
Dipartimento
di Automatica e Informatica

Code a priorità e heap

Gianpiero Cabodi e Paolo Camurati

ADT Heap

Definizione: albero binario con

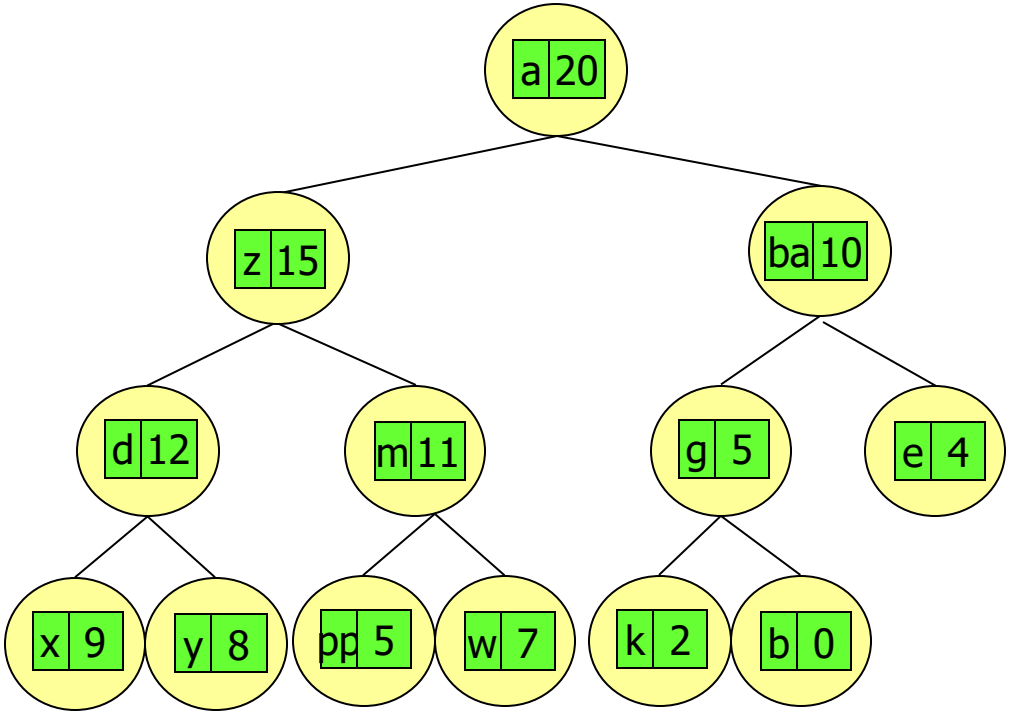
- proprietà strutturale: completo a sinistra (tutti i livelli completi, tranne eventualmente l'ultimo, riempito da SX a DX) $\Rightarrow$ bilanciato
- proprietà funzionale:

$$\forall\ i \neq r \quad key(parent(i)) \geq key(i)$$

Conseguenza: chiave max nella radice

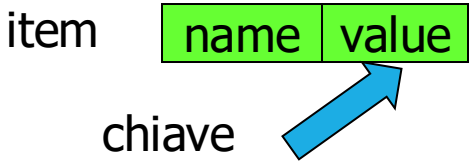
Implementazione: mediante vettore.

Esempio



Item

- Quasi ADT Item
- Dati:
  - Nome (stringa), valore (intero)
  - Chiave = valore
  - Tipologia 3



Operazioni:

- HEAPify(Heap h, int i);
- HEAPbuild(Heap h);
- HEAPsort(Heap h);

Funzioni di servizio

- HEAPinit(int maxN);
- HEAPfree(Heap h);
- HEAPfill(Heap h, Item val);
- HEAPdisplay(Heap h);

ADT di I classe Heap

Heap.h

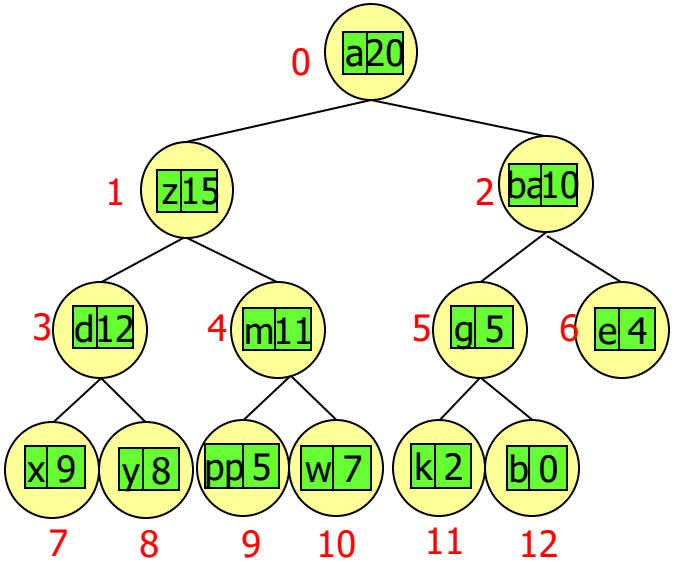
```
typedef struct heap *Heap;

Heap     HEAPinit(int maxN);
Void     HEAPfree(Heap h);
void     HEAPfill(Heap h, Item val);
void     HEAPsort(Heap h);
void     HEAPdisplay(Heap h);
```

Implementazione

- Struttura dati: vettore di Item h->A[0..maxN-1]
- h->heapsize(A): numero di elementi in heap h->A
- radice in h->A[0]
- dato h->A[i]:
  - il figlio SX è h->A[LEFT(i)] dove LEFT(i)= 2i+1
  - il figlio DX è h->A[RIGHT(i)] dove RIGHT(i) = 2i+2
  - il padre è h->A[PARENT(i)] dove PARENT(i)=(i-1)/2

Esempio



maxN = 15

h->A

a	z	ba	d	m	g	e	x	y	pp	w	k	b		
20	15	10	12	11	5	4	9	8	5	7	2	0		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

h->heapsize 13

Heap.c

```
#include <stdio.h>
#include <stdlib.h>
#include "Item.h"
#include "Heap.h"

struct heap { Item *A; int heapsize; };

int LEFT(int i) { return (i*2 + 1); }
int RIGHT(int i) { return (i*2 + 2); }
int PARENT(int i) { return ((i-1)/2); }

Heap HEAPinit(int maxN) {
  Heap h;
  h = malloc(sizeof(*h));
  h->A = malloc(maxN*sizeof(Item));
  h->heapsize = 0;
  return h;
}
```

usata per inserire valori,
non necessariamente
il risultato sarà uno heap

```
void HEAPfree(Heap h) {
  free(h->A);
  free(h);
}

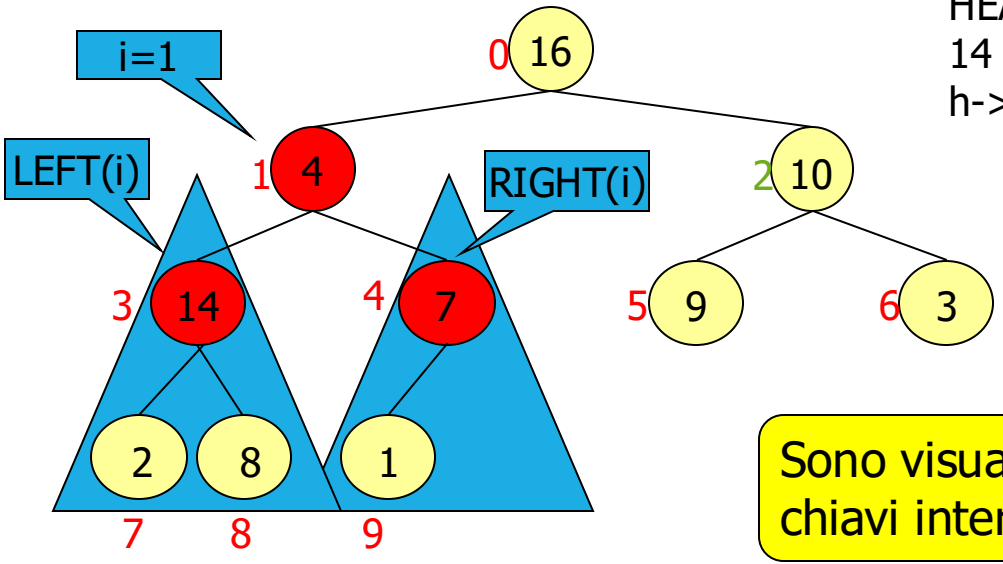
void HEAPfill(Heap h, Item item) {
  int i;
  i  = h->heapsize++;
  h->A[i] = item;
  return;
}

void HEAPdisplay(Heap h) {
  int i;
  for (i = 0; i < h->heapsize; i++)
    ITEMstore(h->A[i]);
}
```

Funzione HEAPify

- Trasforma in heap i, LEFT(i), RIGHT(i), dove LEFT(i) e RIGHT(i) sono già heap
- assegna ad A[i] il max tra A[i], A[LEFT(i)] e A[RIGHT(i)]
- se c'è stato scambio A[i] $\leftrightarrow$ A[LEFT(i)], applica ricorsivamente HEAPify su sottoalbero con radice LEFT(i)
- analogamente per scambio A[i] $\leftrightarrow$ A[RIGHT(i)].
- Complessità: $T(n) = O(\lg n)$.

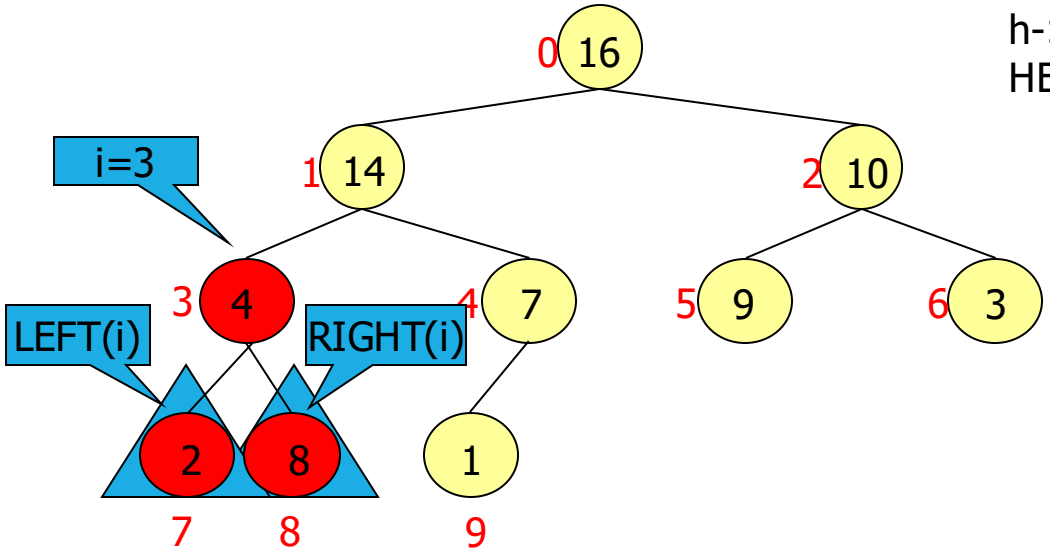
Esempio

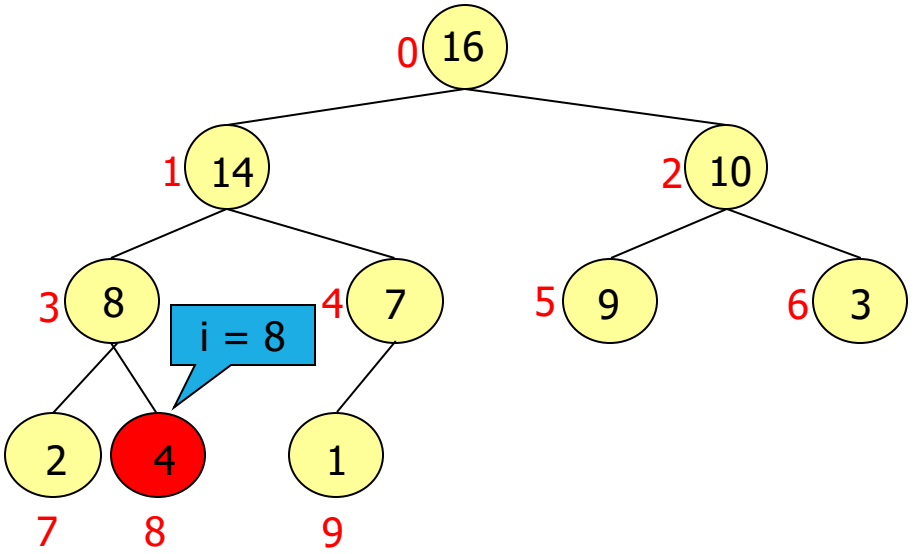


HEAPify(h->A, 1)
14 = max(4, 14, 7)
h->A[i] ↔ h->A[LEFT(i)]

Sono visualizzate solo le chiavi intere, non gli item

HEAPify(h->A, 3)
8 = max(4, 2, 8)
h->A[i] ↔ h->A[RIGHT(i)]
HEAPify(h->A, 8)





HEAPify(h->A, 8)
foglia
terminazione.

```
void HEAPify(Heap h, int i) {
  int l, r, largest;
  l = LEFT(i);
  r = RIGHT(i);
  if ((l<h->heapsize) &&
       KEYcmp(KEYget(h->A[l]),KEYget(h->A[i]))==1)
   largest = l;
  else
    largest = i;
  if ((r<h->heapsize) &&
       KEYcmp(KEYget(h->A[r]), KEYget(h->A[largest]))==1)
    largest = r;
  if (largest != i) {
    Swap(h, i,largest);
    HEAPify(h, largest);
  }
}
```

Funzione HEAPbuild

Trasforma un albero binario memorizzato in vettore A in uno heap:

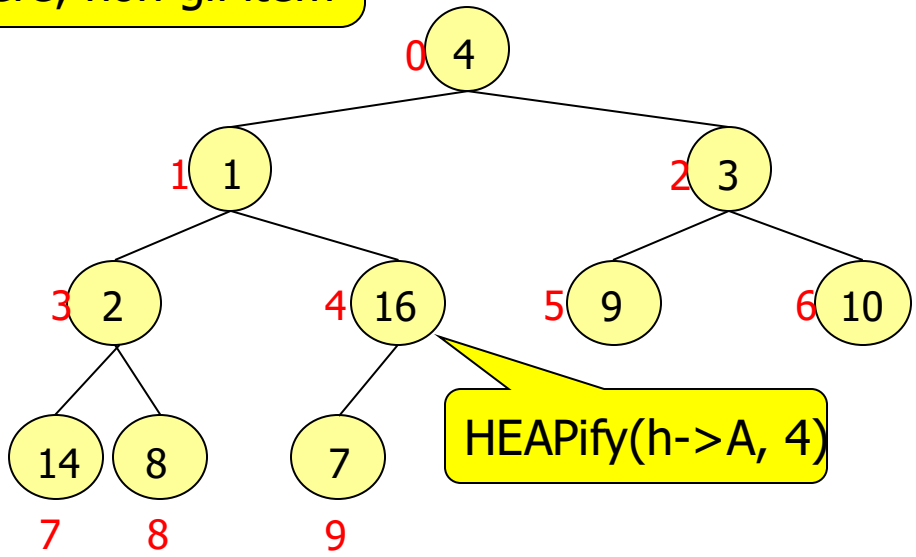
- le foglie sono heap
- applica HEAPify a partire dal padre dell'ultima foglia o coppia di foglie fino alla radice.

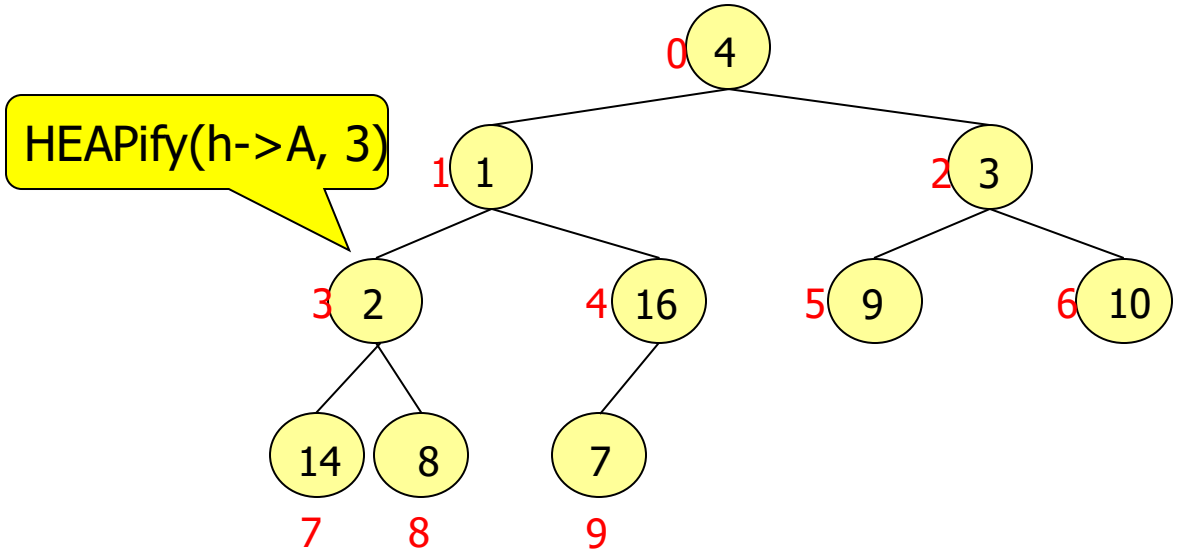
```
void HEAPbuild (Heap h) {
  int i;
  for (i=(h->heapsize)/2-1; i >= 0; i--)
    HEAPify(h, i);
}
```

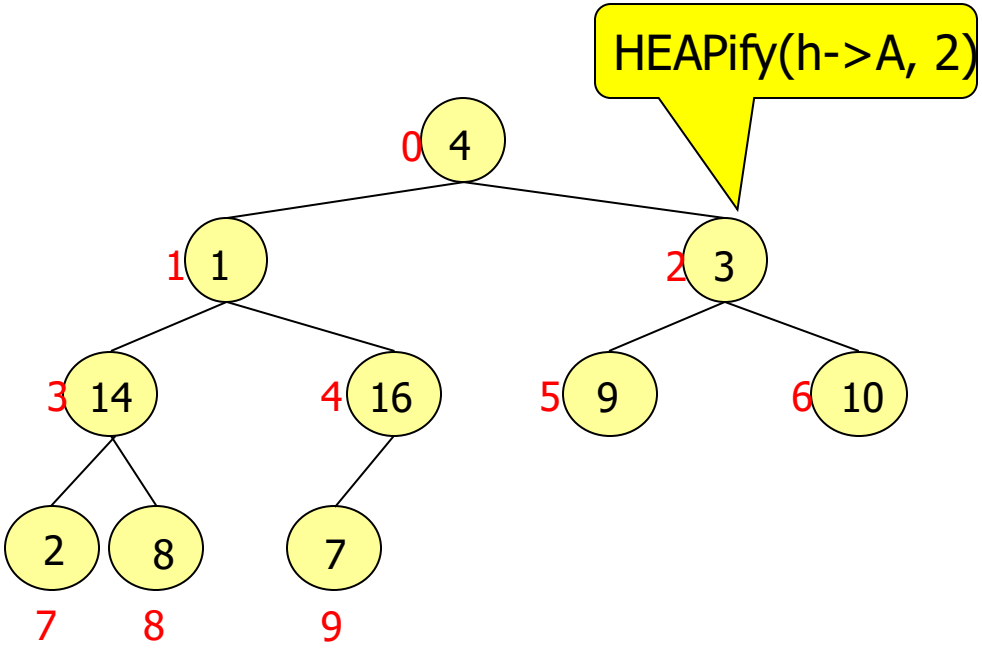
Esempio

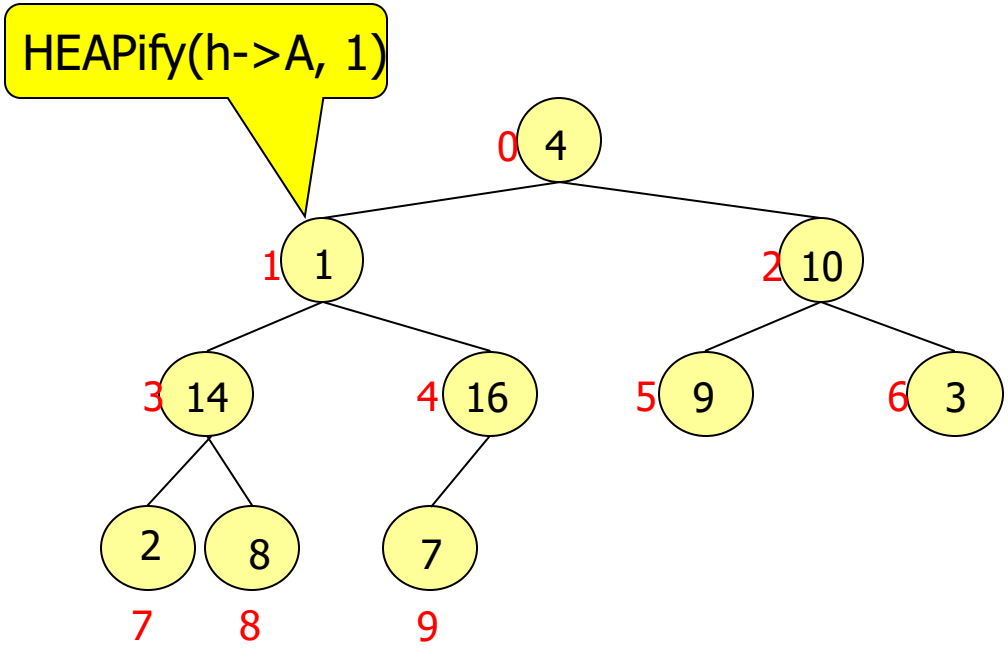
Sono visualizzate solo le chiavi intere, non gli item

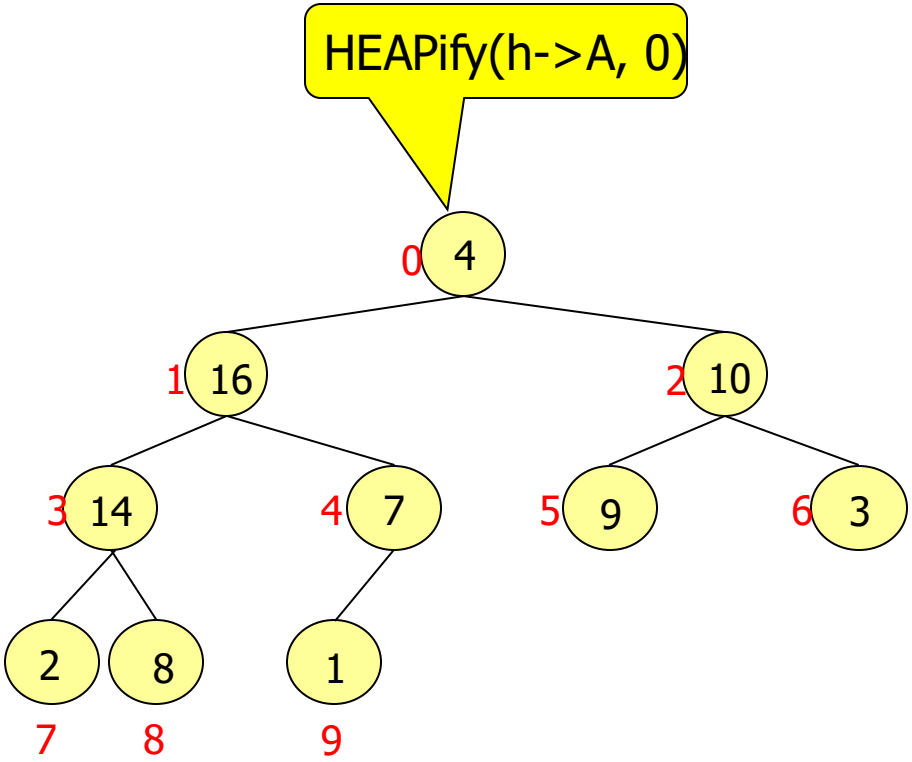
HEAPbuild(h->A)

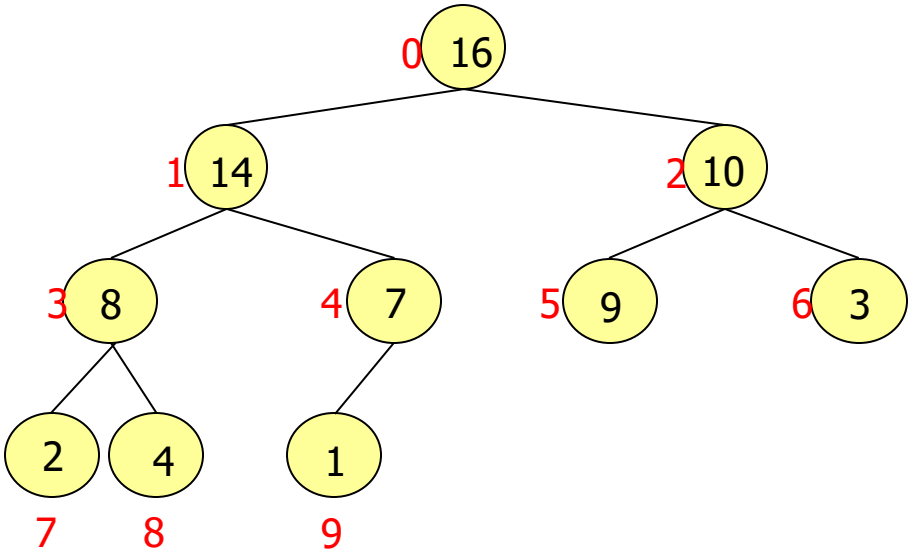



HEAPify(h->A, 3)
0
4
1
1
2
3
3
2
4
16
5
9
6
10
14
8
7
7
8
9


HEAPify(h->A, 2)
0
4
1
1
2
3
3
14
4
16
5
9
6
10
2
8
7
7
8
9


HEAPify(h->A, 1)
0 4
1 1
2 10
3 14
4 16
5 9
6 3
2
8
7
7
8
9


HEAPify(h->A, 0)
0
4
1
16
2
10
3
14
4
7
5
9
6
3
2
8
1
7
8
9


0
16
1
14
2
10
3
8
4
7
5
9
6
3
2
4
1
7
8
9

Analisi di complessità:

- intuitiva ed imprecisa: n passi ciascuno di costo logn, quindi T(n) = O(n lg n)
- precisa: T(n) = O(n).

Risoluzione per sviluppo (unfolding):

$T(n) = 2T(n/2) + \log_2(n)$

$T(n/2) = 2T(n/4) + \log_2(n/2)$

$T(n/4) = 2T(n/8) + \log_2(n/4)$

2 sottoalberi

Heapify

$\sum_{i=0}^{k} ix^i = x\,(1-(k+1)x^k+kx^{k+1})/(1-x)^2$

Sostituendo in T(n):

$T(n) = \sum_{i=0}^{log_2 n} 2^i \log_2(n/2^i)$

$= \log_2 n \sum_{i=0}^{log_2 n} 2^i - \sum_{i=0}^{log_2 n} i2^i$

$= \log_2 n(2n-1) - 2(1-(\log_2 n+1)n+2n\log_2 n)$

$= 2n - \log_2 n - 2$

$= O(n)$

Funzione HEAPsort

- Trasforma il vettore in uno heap mediante HEAPbuild
- Scambia il primo e ultimo elemento
- Riduce la dimensione dello heap di 1
- Ripristina la proprietà di heap
- Ripete fino a esaurimento dello heap.

Caratteristiche:

- complessità: $T(n)= O(n \lg n)$.
- in loco
- non stabile

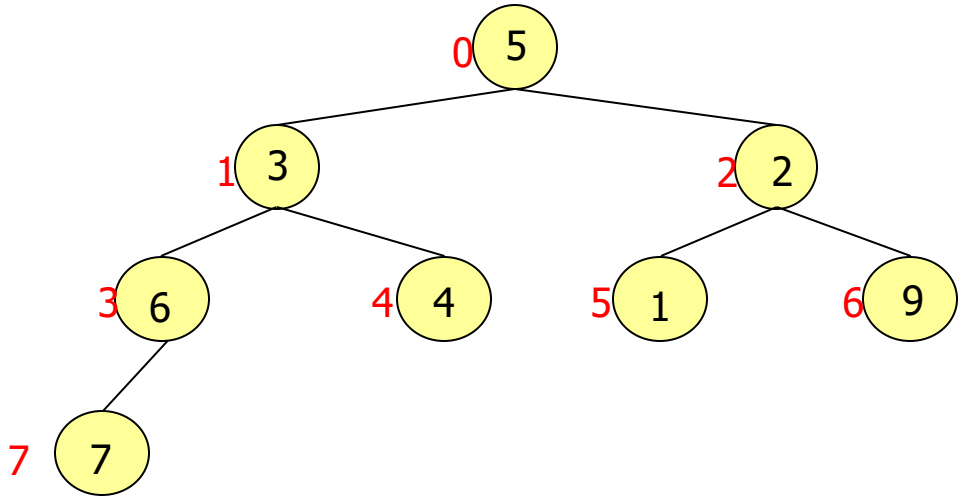
Esempio

Sono visualizzate solo le chiavi intere, non gli item

Configurazione iniziale

h->A

5	3	2	6	4	1	9	7

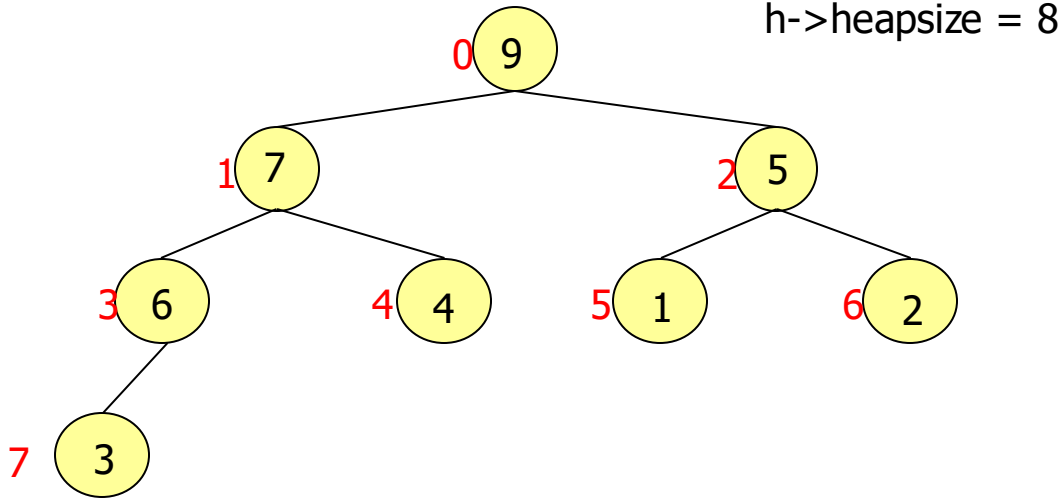


Applico
HEAPbuild

h->A

9	7	5	6	4	1	2	3

h->heapsize = 8

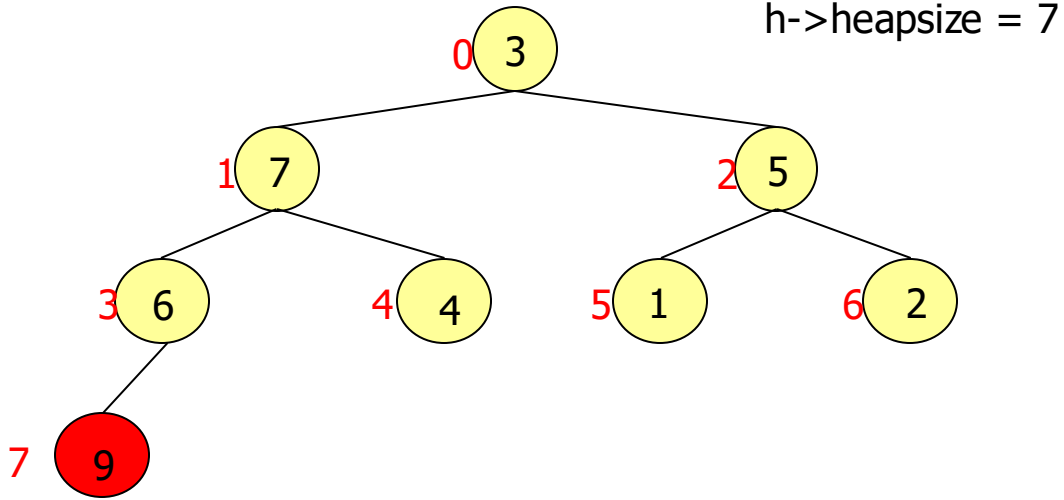


h->A[0] ↔ h->A[h->heapsize-1]
h->heapsize--

h->A

3	7	5	6	4	1	2	9

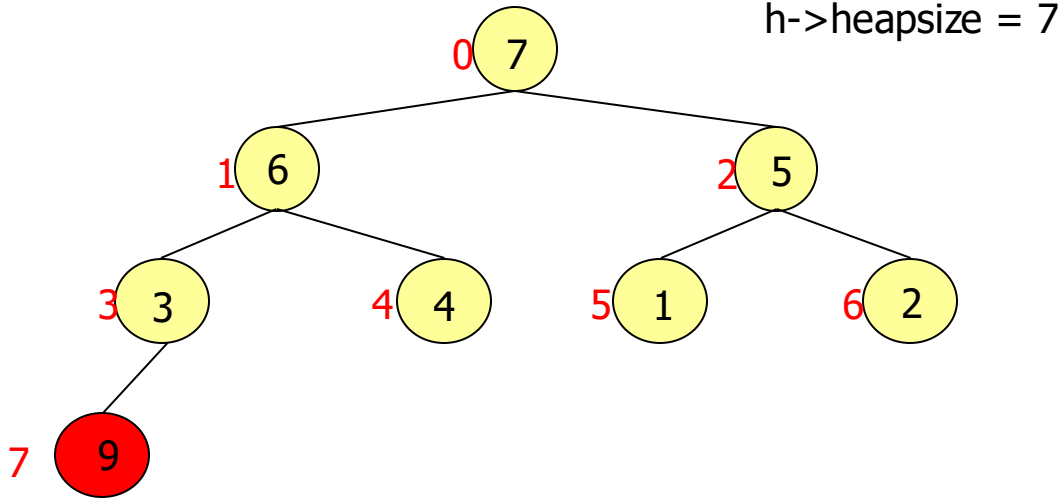
h->heapsize = 7



HEAPify(h->A, 0)



h->heapsize = 7

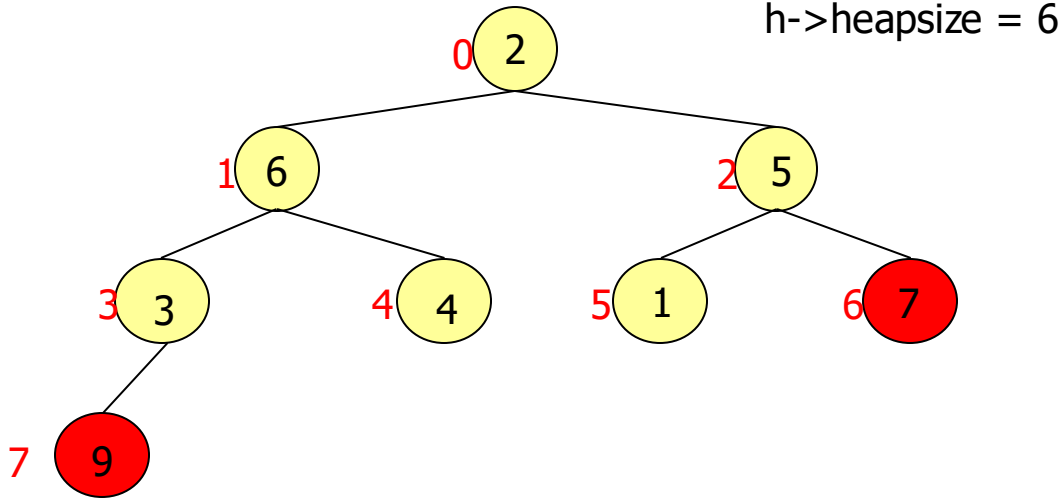


h->A[0] ↔ h->A[h->heapsize-1]
h->heapsize--

h->A

2	6	5	3	4	1	7	9

h->heapsize = 6

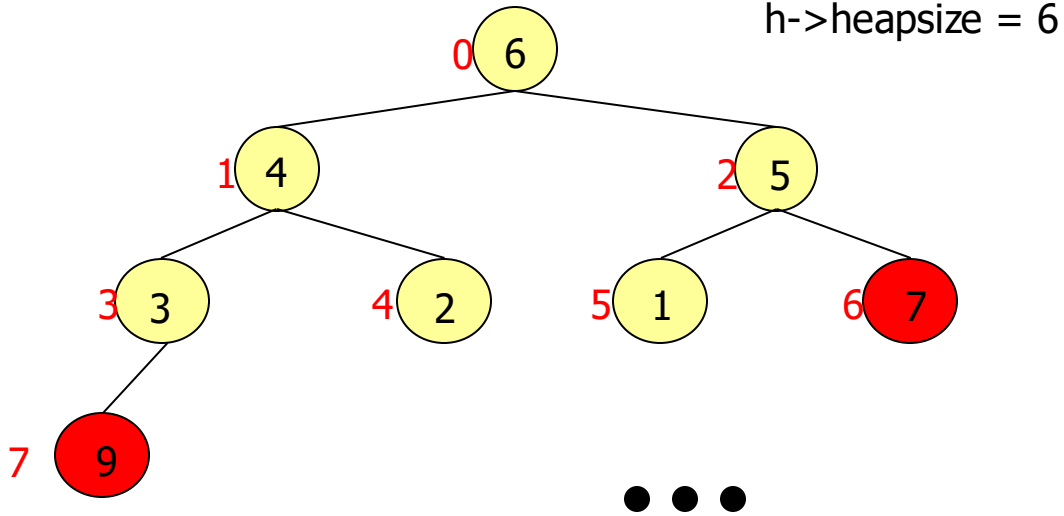


HEAPify(h->A, 0)

h->A

6	4	5	3	2	1	7	9

h->heapsize = 6



```
void HEAPsort(Heap h) {
  int i, j;
  HEAPbuild(h);
  j = h->heapsize;
  for (i = h->heapsize-1; i > 0; i--) {
    Swap (h,0,i);
    h->heapsize--;
    HEAPify(h,0);
  }
  h->heapsize = j;
}
```

Coda a priorità

Definizione:

- struttura dati PQ per mantenere un set di elementi di tipo Item, ciascuno dei quali include un campo priorità
- operazioni principali: inserzione, estrazione del massimo, lettura del massimo, cambio di priorità.

ADT di I classe Coda a priorità

PQ.h

```
typedef struct pqueue *PQ;

PQ      PQinit(int maxN);
void    PQfree(PQ pq);
int     PQempty(PQ pq);
void    PQinsert(PQ pq, Item val);
Item    PQextractMax(PQ pq);
Item    PQshowMax(PQ pq);
void    PQdisplay(PQ pq);
int     PQsize(PQ pq);
void    PQchange(PQ pq, Item val);
```

discussione a parte

Implementazione della struttura dati:

- vettore/lista non ordinato
- vettore/lista ordinato
- heap di dati/indici.

} non considerati qui, cfr Tipi di Dato Astratto

Complessità:

	PQinsert	PQshowMax	PQextractMax
Vettore non ordinato	1	N	N
Lista non ordinata	1	N	N
Vettore ordinato	N	1	1
Lista ordinata	N	1	1
Heap di item/indici	logN	1	logN

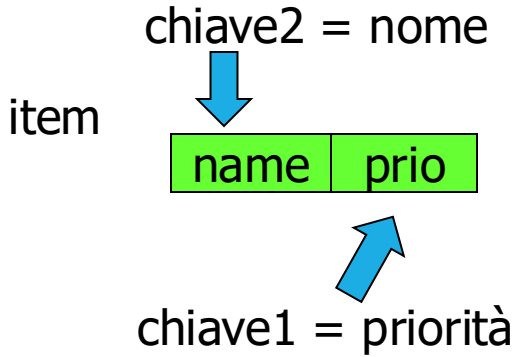
max in coda

max in testa

Cosa contiene l'ADT Coda a priorità?

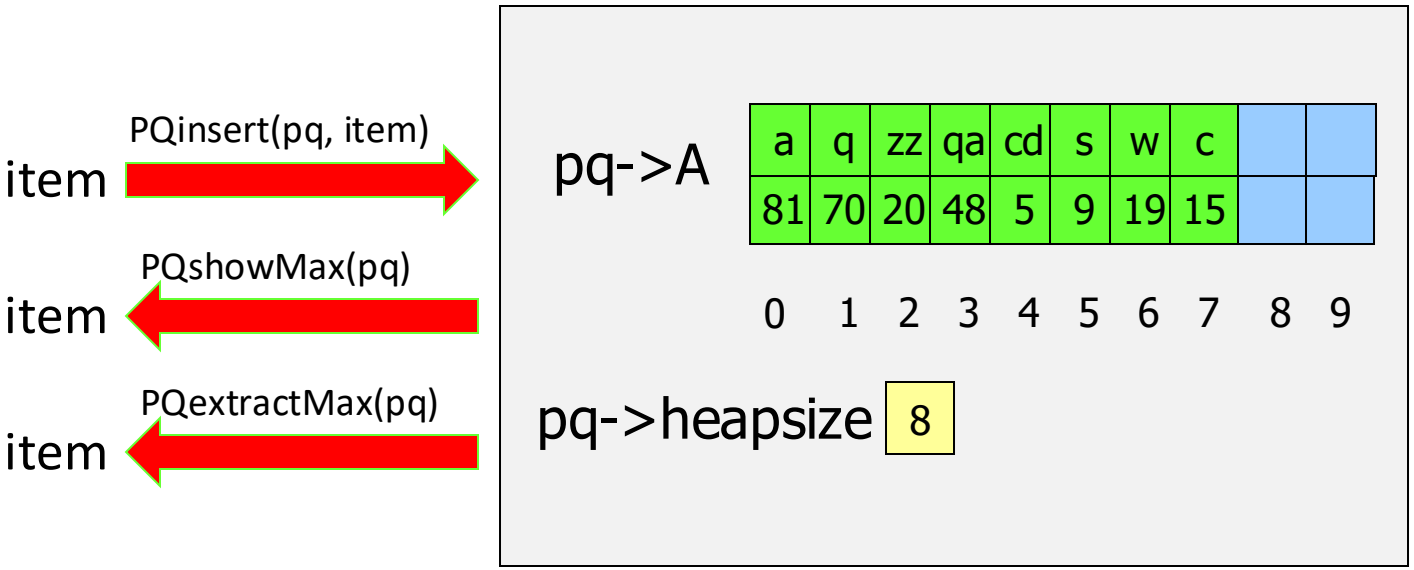
I soluzione: la coda a priorità contiene dati (lo heap che realizza la coda a priorità contiene i dati), l'ADT è una struct con:

1. la coda a priorità: vettore (heap) pq->A di dati di tipo Item (quasi ADT, tipologia 3)
2. heapsize: intero.



Utente

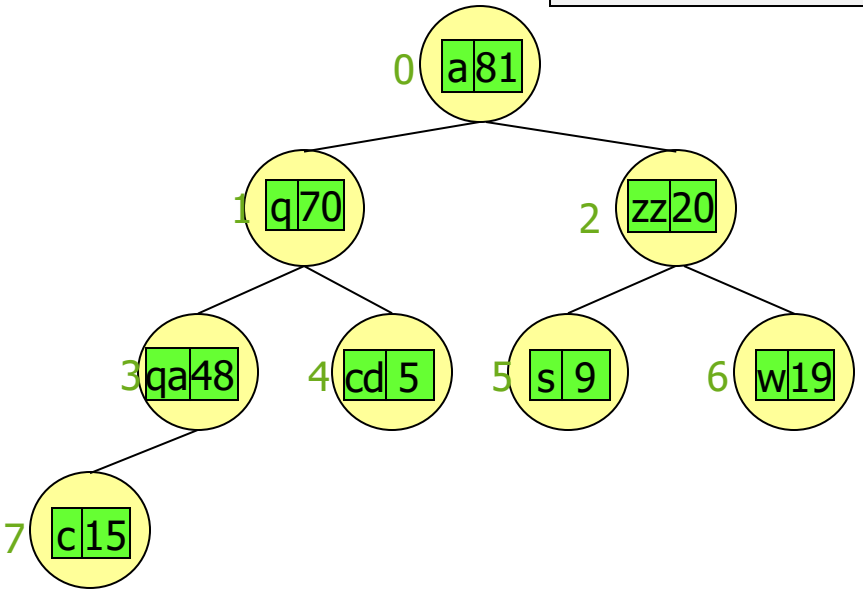
ADT I cat. coda a priorità di dati



pq->A

a	q	zz	qa	cd	s	w	c		
81	70	20	48	5	9	19	15		
0	1	2	3	4	5	6	7	8	9

pq->heapsize 8



ADT di I classe Coda a priorità di dati

PQ.c

```
#include <stdlib.h>
#include "Item.h"
#include "PQ.h"

struct pqueue { Item *A; int heapsize; };

static int LEFT(int i) { return (i*2 + 1); }
static int RIGHT(int i) { return (i*2 + 2); }
static int PARENT(int i) { return ((i-1)/2); }

PQ PQinit(int maxN){
  PQ pq = malloc(sizeof(*pq));
  pq->A = (Item *)malloc(maxN*sizeof(Item));
  pq->heapsize = 0;
  return pq;
}
```

```
void PQfree(PQ pq){
  free(pq->A);
  free(pq);
}

int PQempty(PQ pq) { return pq->heapsize == 0; }

int PQsize(PQ pq) { return pq->heapsize; }

Item PQshowMax(PQ pq) { return pq->A[0]; }

void PQdisplay(PQ pq) {
  int i;
  for (i = 0; i < pq->heapsize; i++)
    ITEMstore(pq->A[i]);
}
```

Funzione PQinsert

- Aggiunge una foglia all'albero (cresce per livelli da SX a DX, rispettando la proprietà strutturale)
- Risale dal nodo corrente (inizialmente la foglia appena creata) fino al più alla radice. Confronta la chiave del dato contenuto nel padre con la chiave del dato da inserire, facendo scendere il dato del padre nel figlio se la chiave da inserire è maggiore, altrimenti inserisce il dato nel nodo corrente.

Complessità: $T(n) = O(\lg n)$.

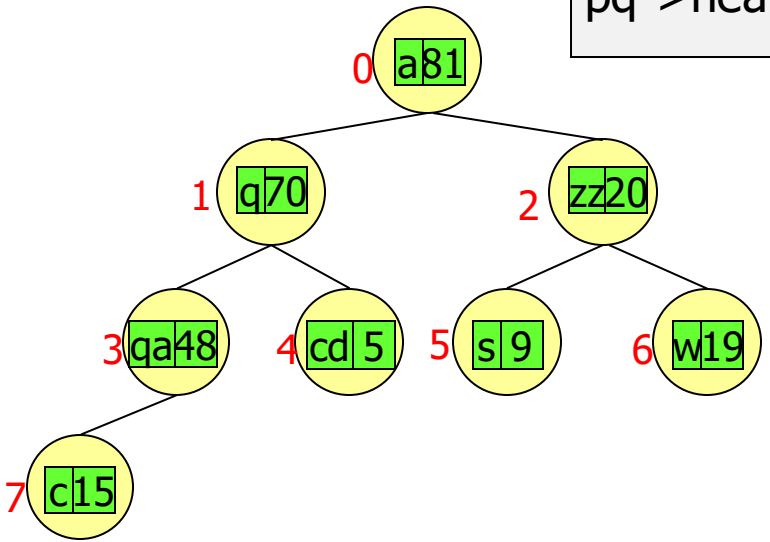
```
void PQinsert (PQ pq, Item val) {
  int i;
  i  = pq->heapsize++;
  while((i>=1) &&
      (KEYcmp(KEYget(pq->A[PARENT(i)]),KEYget(val))==-1)){
    pq->A[i] = pq->A[PARENT(i)];
    i = (i-1)/2;
  }
  pq->A[i] = val;
  return;
}
```

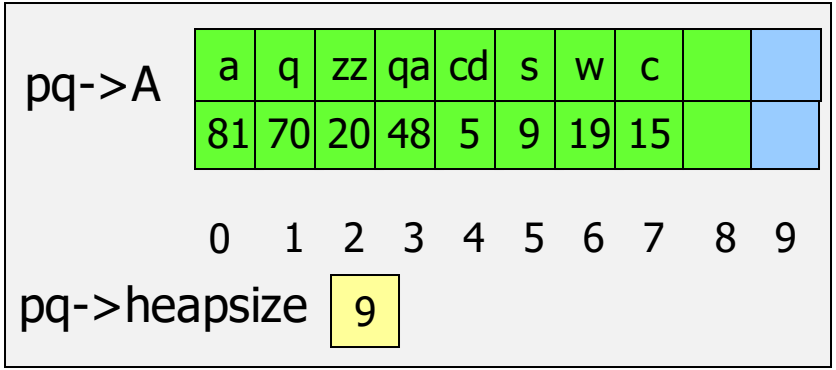
Esempio

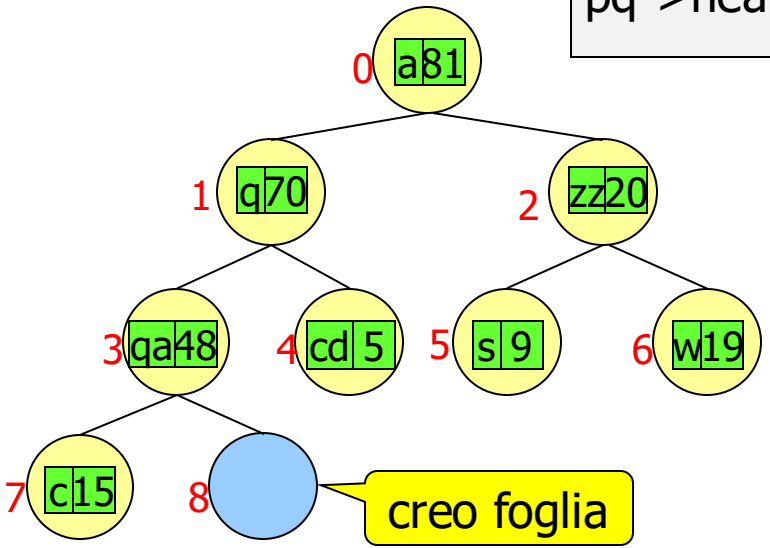
Inserzione di r 75

pq->A	a	q	zz	qa	cd	s	w	c		
	81	70	20	48	5	9	19	15		
	0	1	2	3	4	5	6	7	8	9

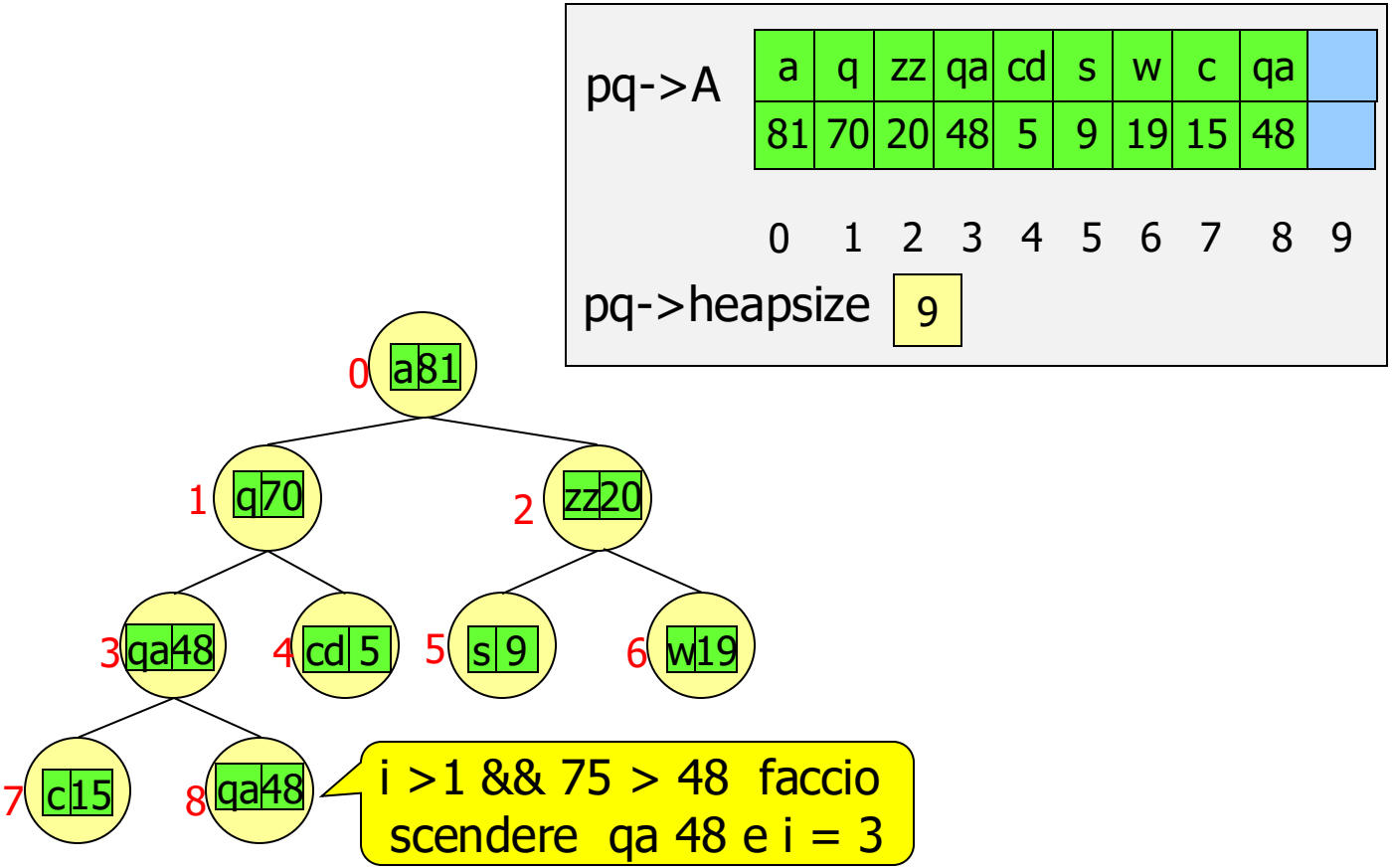
pq->heapsize 8

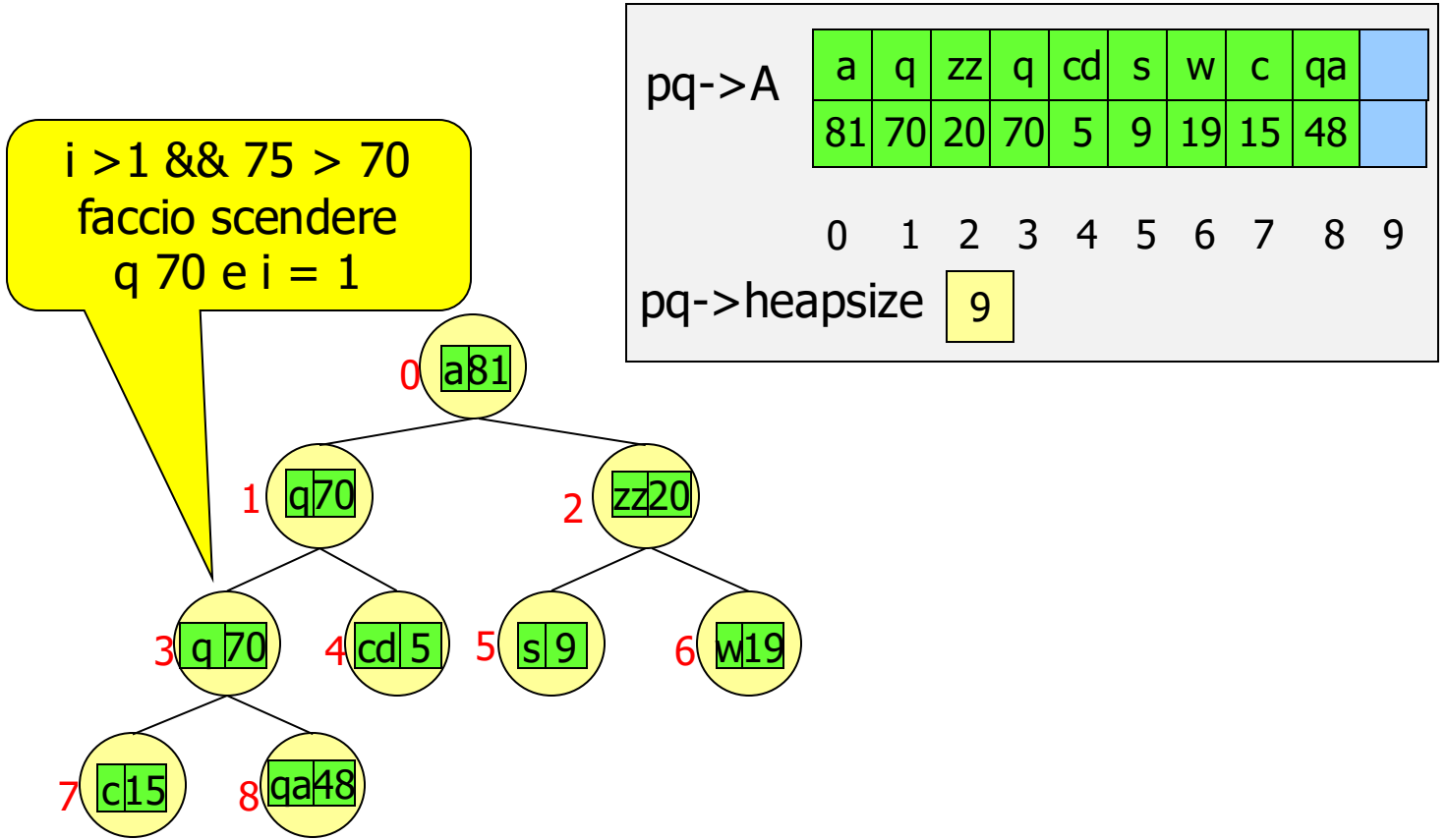


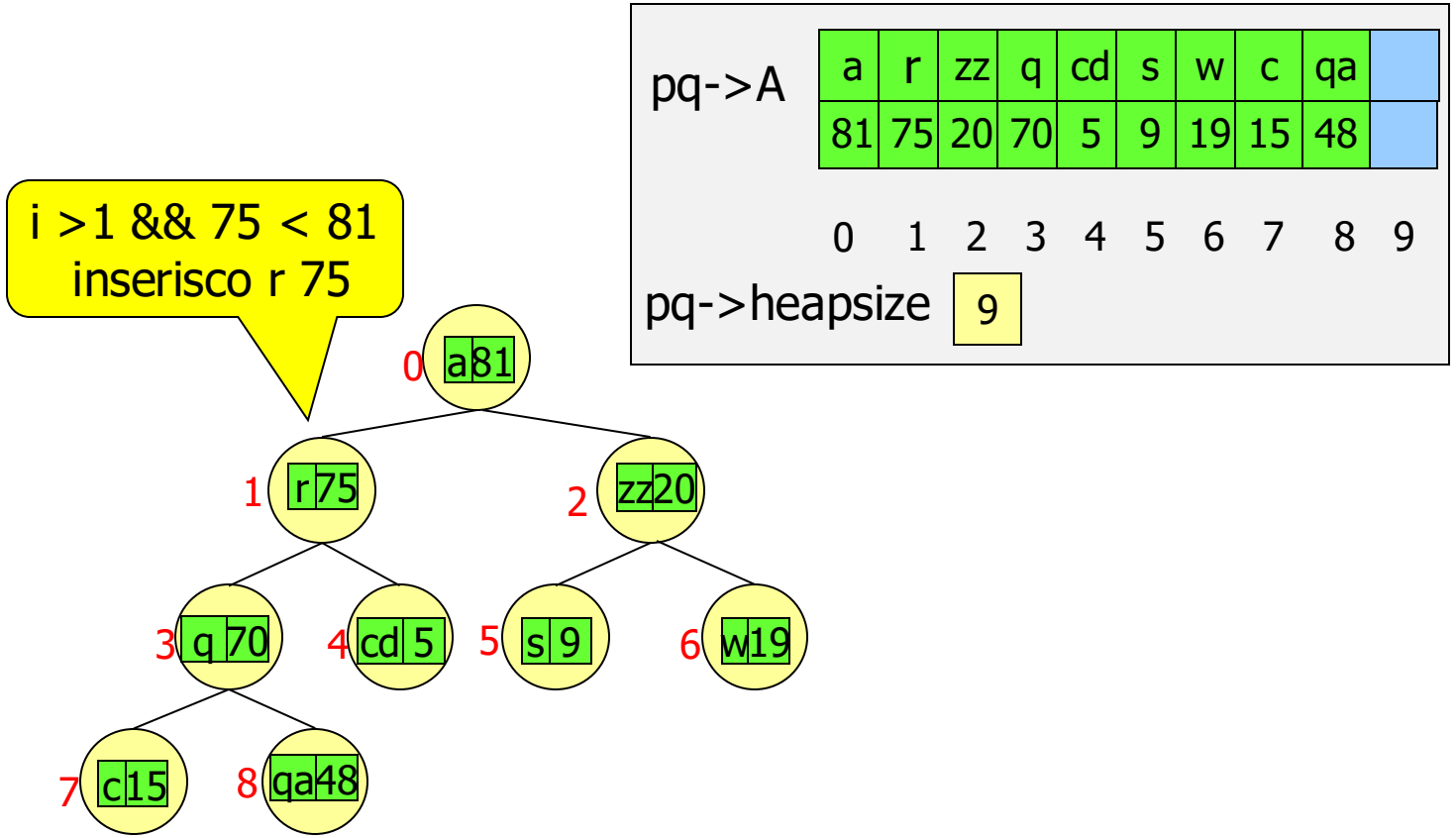

pq->A
a q zz qa cd s w c
81 70 20 48 5 9 19 15
0 1 2 3 4 5 6 7 8 9
pq->heapsize
9


0 a 81
1 q 70
2 zz 20
3 qa 48
4 cd 5
5 s 9
6 w 19
7 c 15
8
creo foglia

i = 9


pq->A
a q zz qa cd s w c qa
81 70 20 48 5 9 19 15 48
0 1 2 3 4 5 6 7 8 9
pq->heapsize
9
0 a81
1 q70
2 zz20
3 qa48
4 cd 5
5 s 9
6 w19
7 c15
8 qa48
i >1 && 75 > 48 faccio
scendere qa 48 e i = 3


pq->A
a q zz q cd s w c qa
81 70 20 70 5 9 19 15 48
0 1 2 3 4 5 6 7 8 9
pq->heapsize
9
i >1 && 75 > 70
faccio scendere
q 70 e i = 1
0 a 81
1 q 70
2 zz 20
3 q 70
4 cd 5
5 s 9
6 w 19
7 c 15
8 qa 48


pq->A
a r zz q cd s w c qa
81 75 20 70 5 9 19 15 48
0 1 2 3 4 5 6 7 8 9
pq->heapsize
9
i >1 && 75 < 81
inserisco r 75
0 a 81
1 r 75
2 zz 20
3 q 70
4 cd 5
5 s 9
6 w 19
7 c 15
8 qa 48

Funzione PQextractMax

- Modifica lo heap, estraendone il valore massimo, che è contenuto nella radice:
- scambia la radice con l'ultima delle foglie (quella più a destra nell'ultimo livello)
- riduce di 1 della dimensione dello heap
- ripristina le proprietà dello heap mediante applicazione di HEAPify.

Complessità: $T(n) = O(\lg n)$.

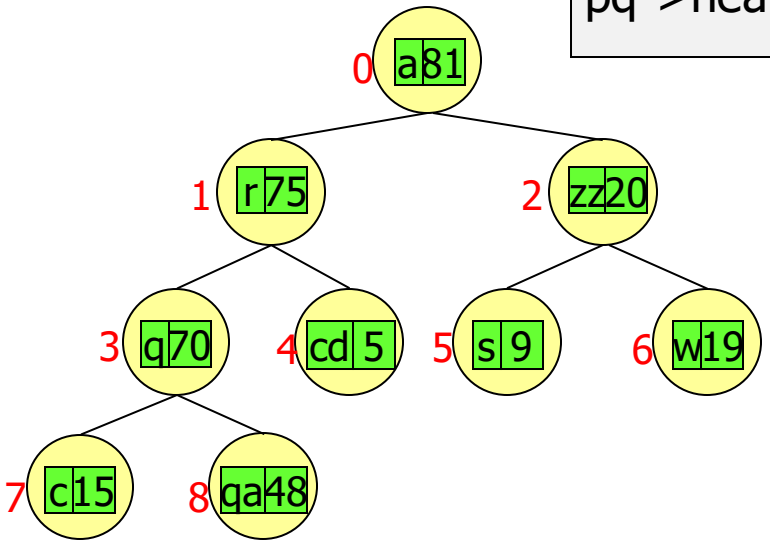
```
Item PQextractMax(PQ pq) {
  Item val;
  Swap (pq, 0,pq->heapsize-1);
  val = pq->A[pq->heapsize-1];
  pq->heapsize--;
  HEAPify(pq, 0);
  return val;
}
```

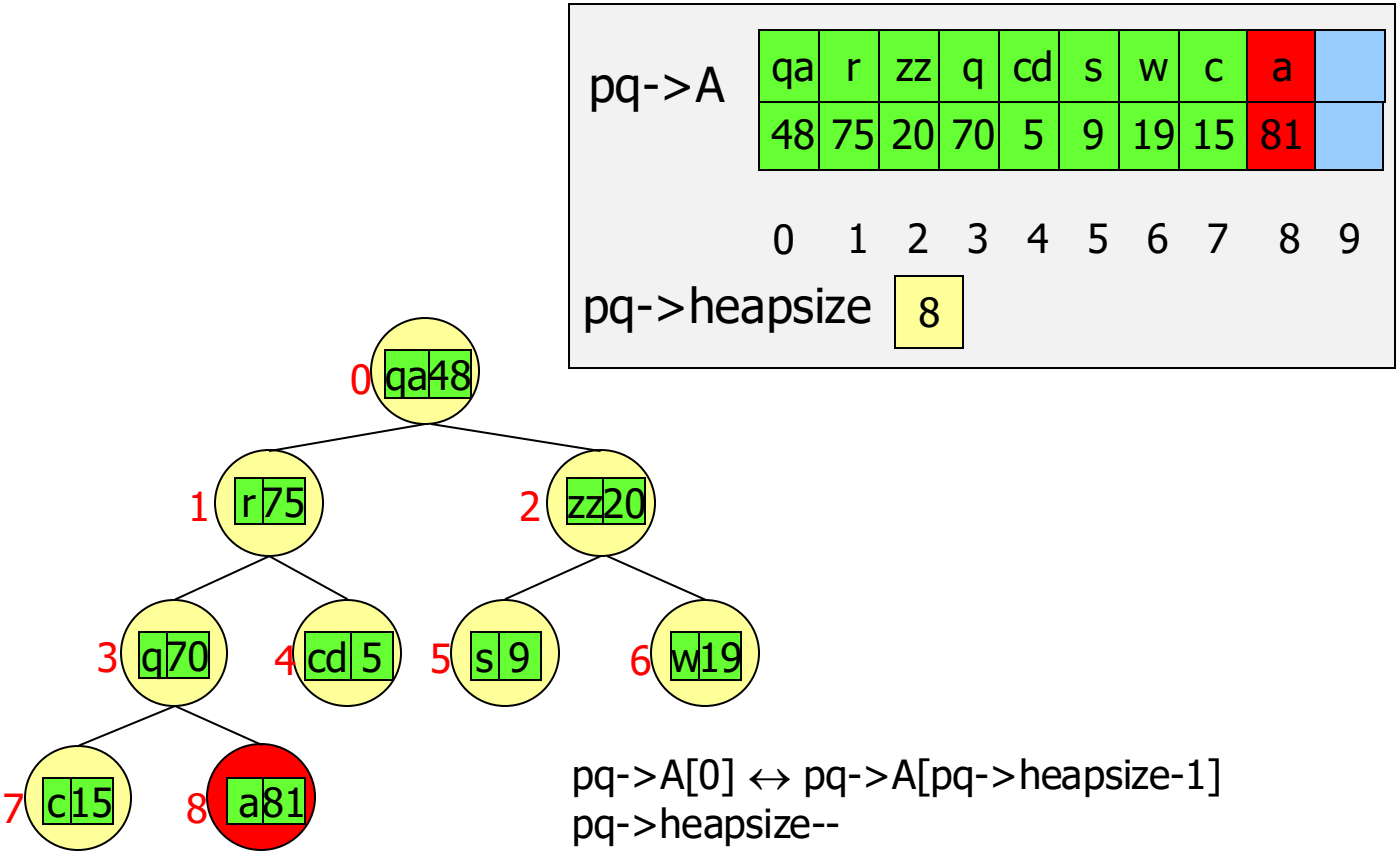
Esempio

pq->A

a	r	zz	q	cd	s	w	c	qa	
81	75	20	70	5	9	19	15	48	
0	1	2	3	4	5	6	7	8	9

pq->heapsize 9



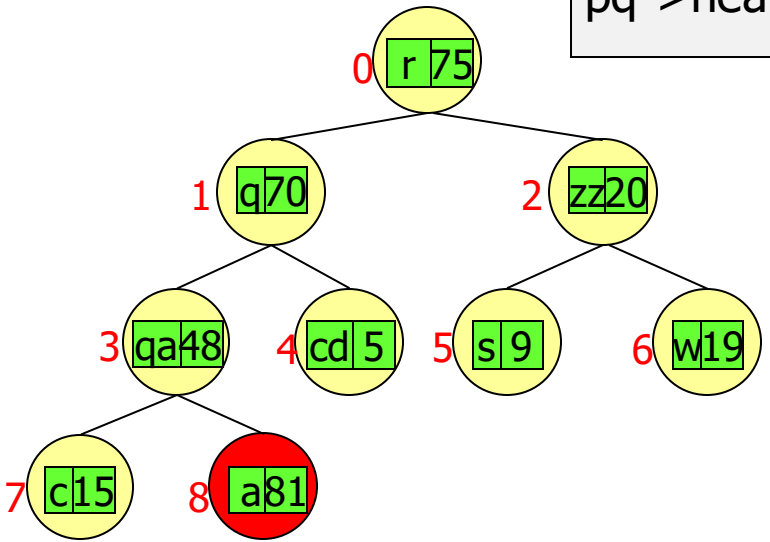

pq->A
qa r zz q cd s w c a
48 75 20 70 5 9 19 15 81
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
0 qa 48
1 r 75
2 zz 20
3 q 70
4 cd 5
5 s 9
6 w 19
7 c 15
8 a 81
pq->A[0] ↔ pq->A[pq->heapsize-1]
pq->heapsize--

HEAPify(pq->A, 0)

pq->A

r	q	zz	qa	cd	s	w	c	a	
75	70	20	48	5	9	19	15	81	
0	1	2	3	4	5	6	7	8	9

pq->heapsize 8



Funzione PQchange

- Modifica la priorità di un elemento, la cui **posizione** (indice nello heap) viene calcolata con una scansione di costo lineare
- o risale dalla posizione data fino al più alla radice confrontando la chiave del padre con la chiave modificata, facendo scendere la chiave del padre nel figlio se la chiave modificata è maggiore, altrimenti la inserisce nel nodo corrente
- o applica HEAPify a partire dalla posizione data.

fixup

fixdown

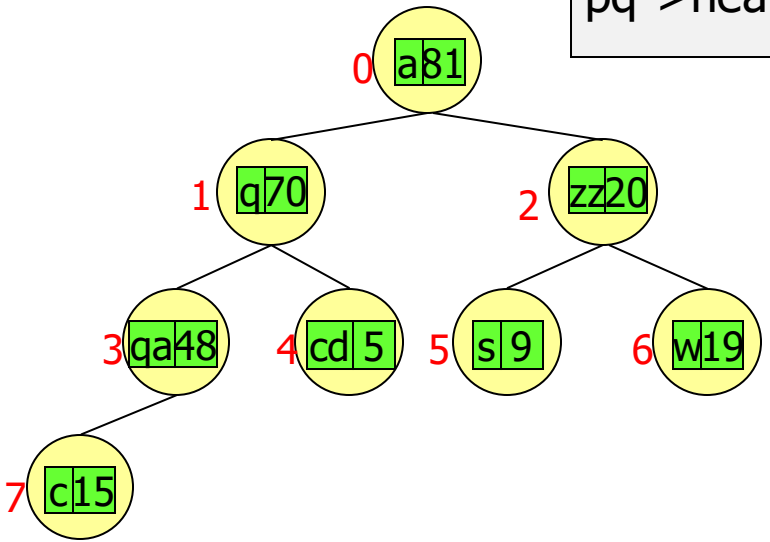
Complessità: $T(n) = O(n) + O(\lg n) = O(n)$.

Esempio

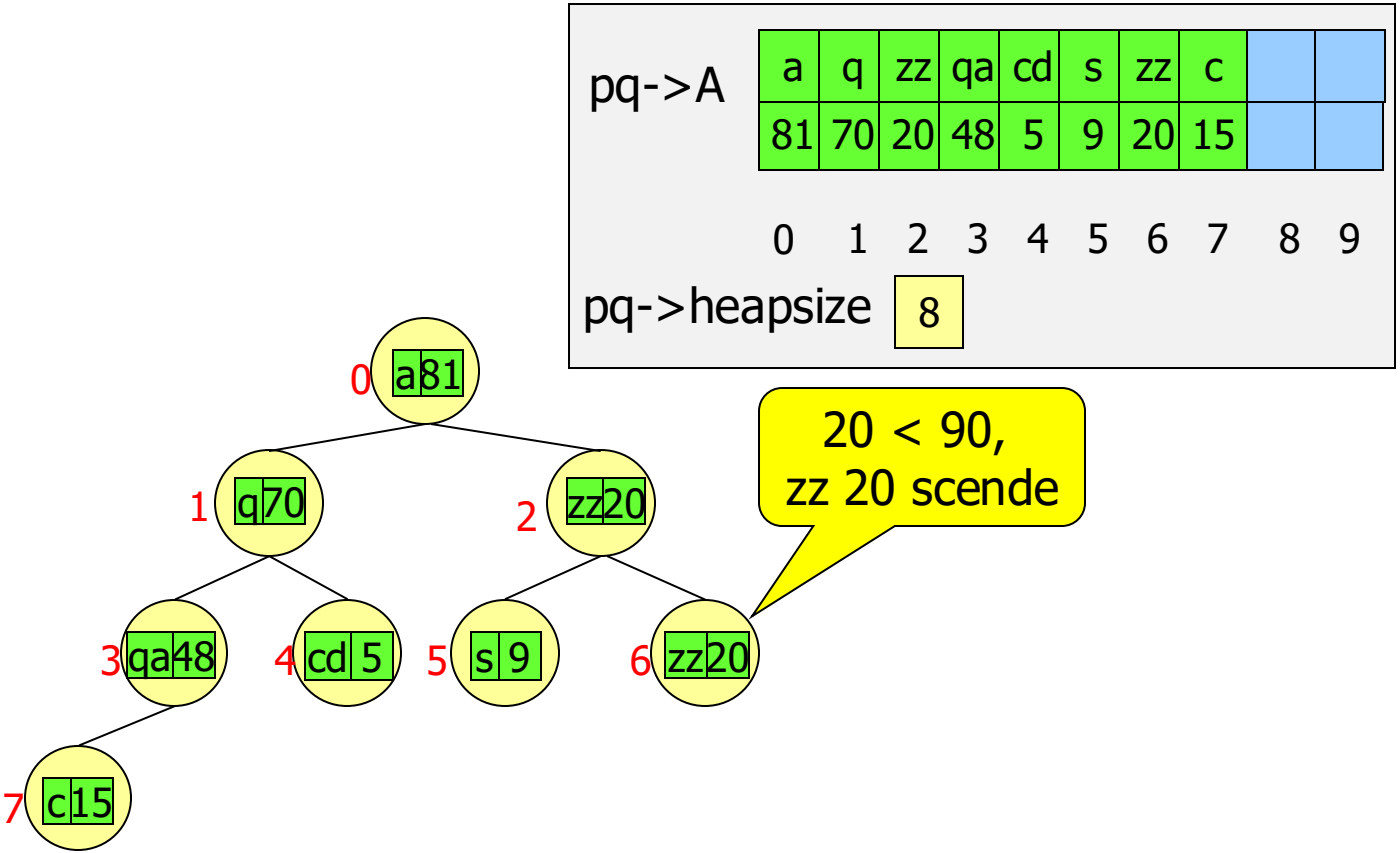
pq->A

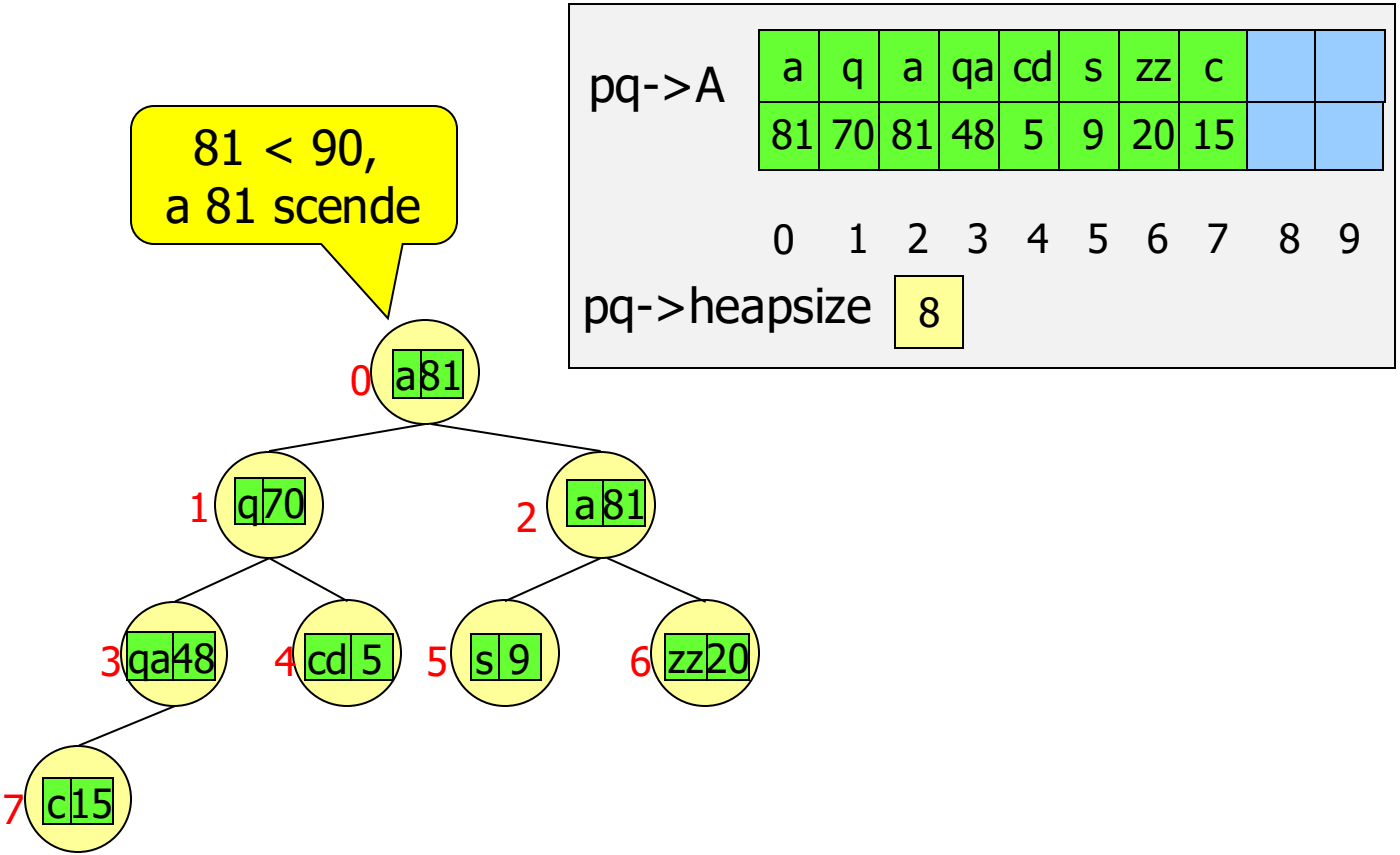
a	q	zz	qa	cd	s	w	c		
81	70	20	48	5	9	19	15		
0	1	2	3	4	5	6	7	8	9

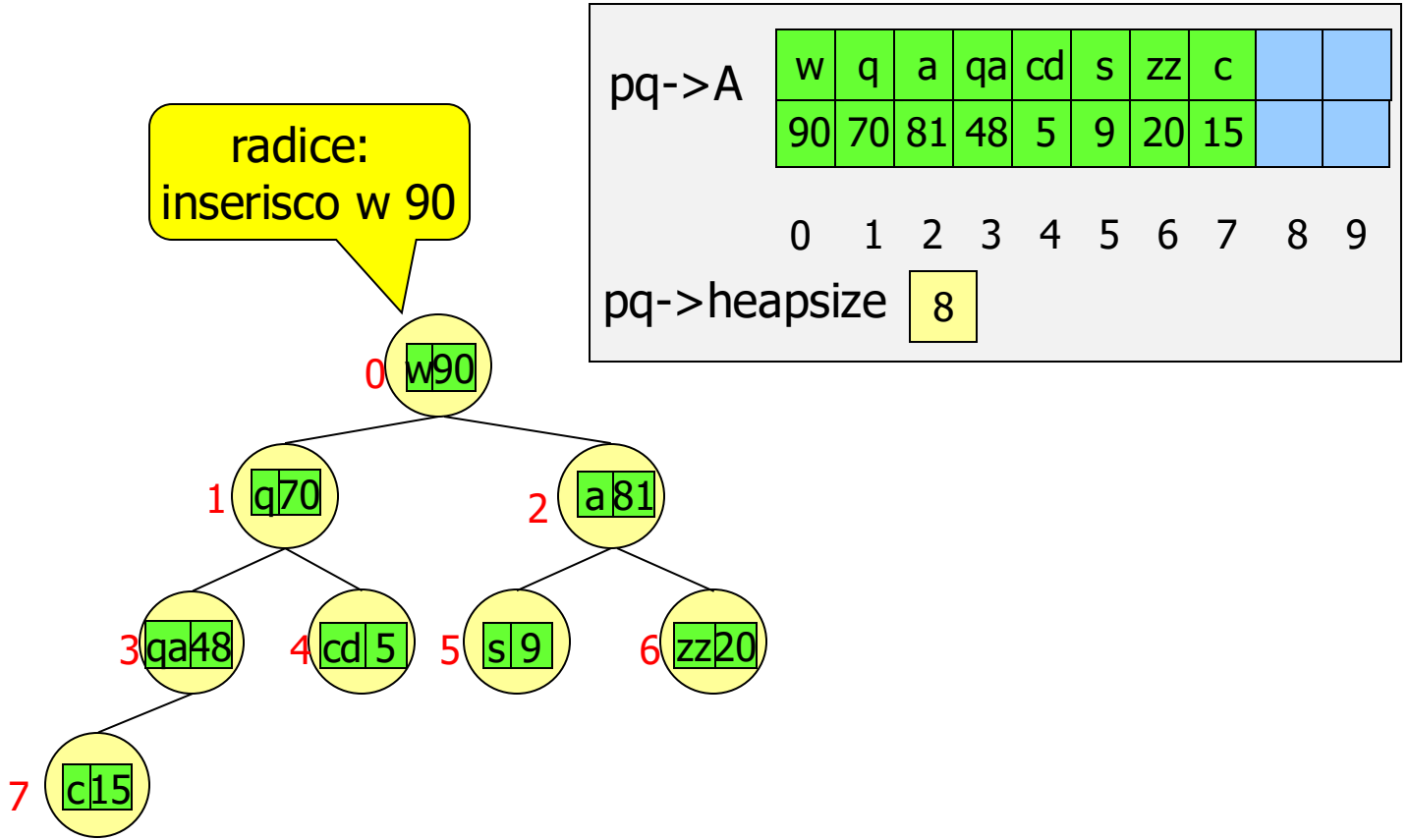
pq->heapsize 8



Cambio la priorità di w da 19 a 90. L'elemento si trova all'indice 6.


pq->A
a q zz qa cd s zz c
81 70 20 48 5 9 20 15
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
0 a 81
1 q 70
2 zz 20
3 qa 48
4 cd 5
5 s 9
6 zz 20
7 c 15
20 < 90,
zz 20 scende


81 < 90,
a 81 scende
pq->A
a q a qa cd s zz c
81 70 81 48 5 9 20 15
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
0 a 81
1 q 70
2 a 81
3 qa 48
4 cd 5
5 s 9
6 zz 20
7 c 15

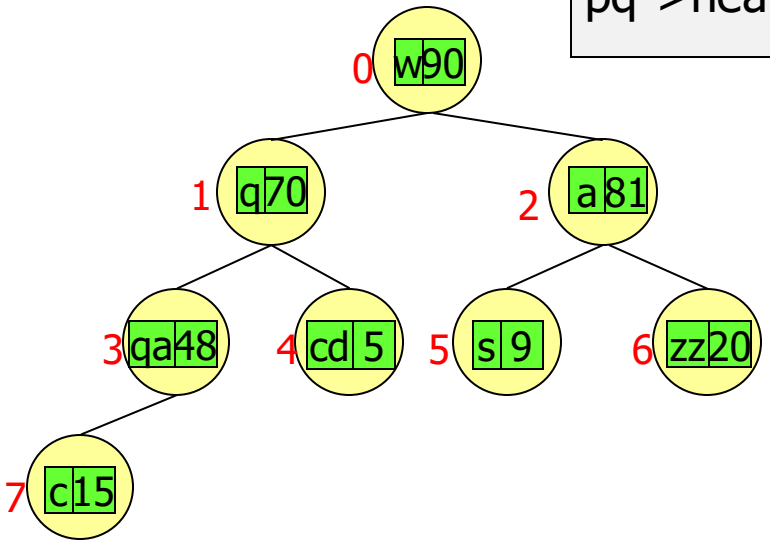

radice:
inserisco w 90
pq->A
w q a qa cd s zz c
90 70 81 48 5 9 20 15
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
0 w 90
1 q 70
2 a 81
3 qa 48
4 cd 5
5 s 9
6 zz 20
7 c 15

Esempio

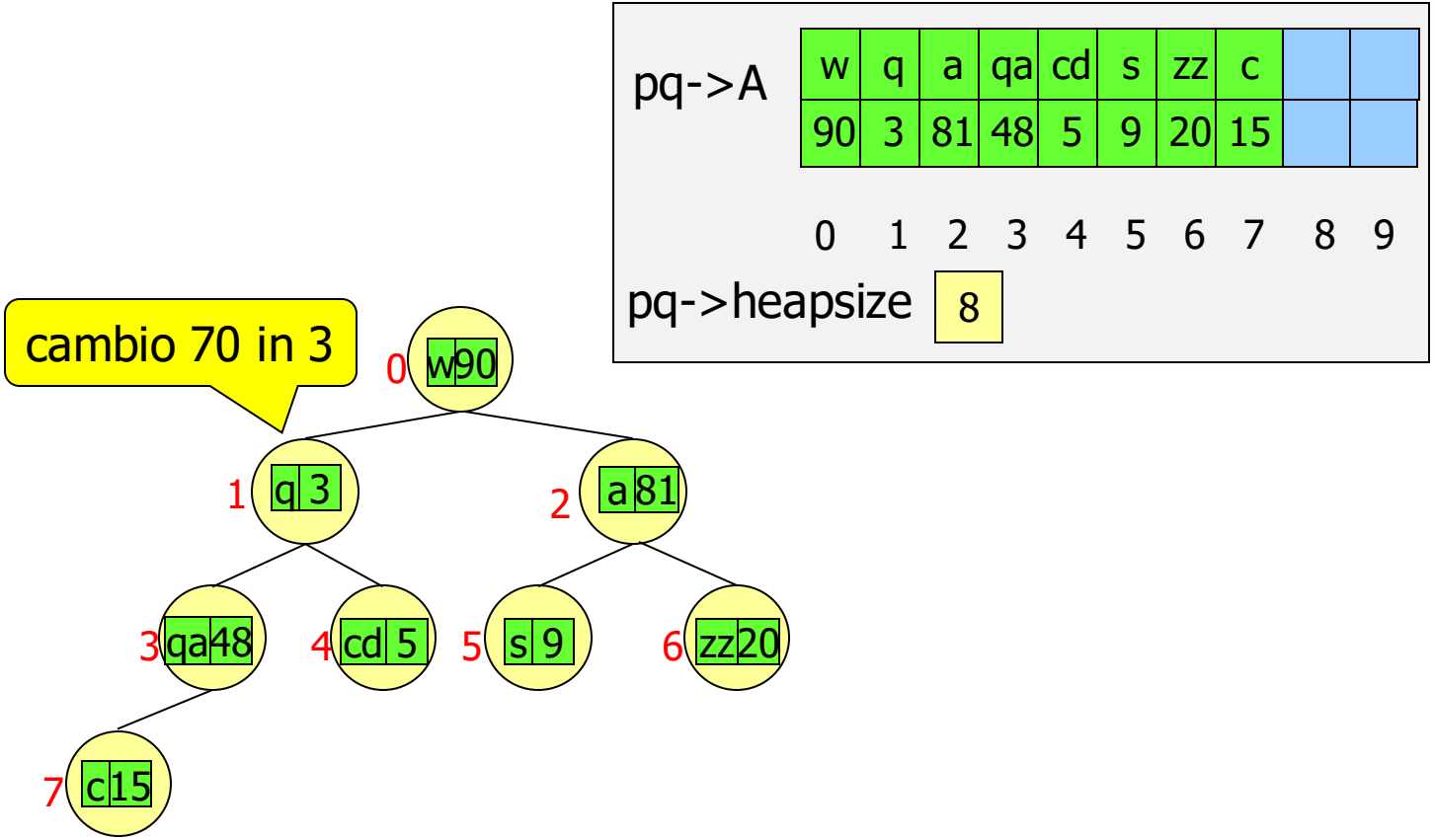
pq->A

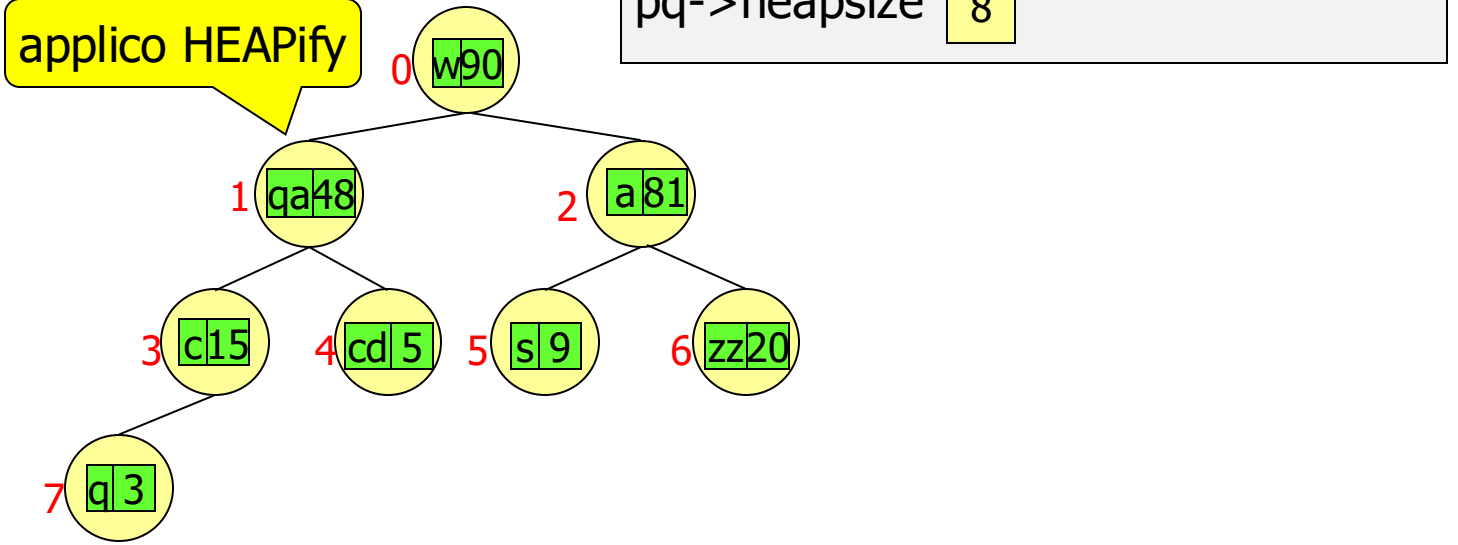
w	q	a	qa	cd	s	zz	c		
90	70	81	48	5	9	20	15		
0	1	2	3	4	5	6	7	8	9

pq->heapsize 8



Cambio la priorità di q da 70 a 3. L'elemento si trova all'indice 1.


pq->A
w q a qa cd s zz c
90 3 81 48 5 9 20 15
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
cambio 70 in 3
0 w 90
1 q 3
2 a 81
3 qa 48
4 cd 5
5 s 9
6 zz 20
7 c 15


pq->A
w qa a c cd s zz q
90 48 81 15 5 9 20 3
0 1 2 3 4 5 6 7 8 9
pq->heapsize
8
applico HEAPify
0 w 90
1 qa 48
2 a 81
3 c 15
4 cd 5
5 s 9
6 zz 20
7 q 3

```
void PQchange (PQ pq, Item val) {
  int i, found = 0, pos;
  for (i = 0; i < pq->heapsize && found == 0; i++)
    if (NAMEcmp(NAMEget(&(pq->A[i])), NAMEget(&val))==0) {
      found = 1;
      pos = i;
    }

  if (found==1) {
    while(pos>=1 && PRIOget(pq->A[PARENT(pos)])<PRIOget(val)){
      pq->A[pos] = pq->A[PARENT(pos)];
      pos = (pos-1)/2;
    }
    pq->A[pos] = val;
    Heapify(pq, pos);
  }
  else
    printf("key not found!\n");
  return;
}
```

È possibile una PQchange di complessità minore?

La complessità della PQchange dipende dalla ricerca dell'item nella coda.

Occorre fare in modo che NON si debba **cercare** l'item nella coda.

Soluzione: deve essere noto «dove» l'item si trova nella coda (gestito dalle operazioni su PQ). Vi sono diverse alternative per localizzare l'item:

1. la PQ non contiene gli item (dati) ma puntatori agli item
2. la PQ non contiene gli item (dati) ma indici che permettono di reperire il dato. Se la chiave dell'item è univoca, la si usa come riferimento tramite una tabella di simboli efficiente.



Soluzione #1

- in coda si inseriscono solo «riferimenti» ad item (es. puntatori)
- l'item ha un campo pos (posizione in PQ)
- il modulo Item fornisce le operazioni ITEMsetPos e ITEMgetPos che permettono di ottenere la posizione di un item con costo O(1)

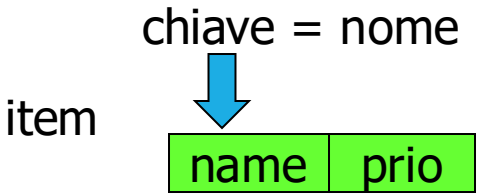


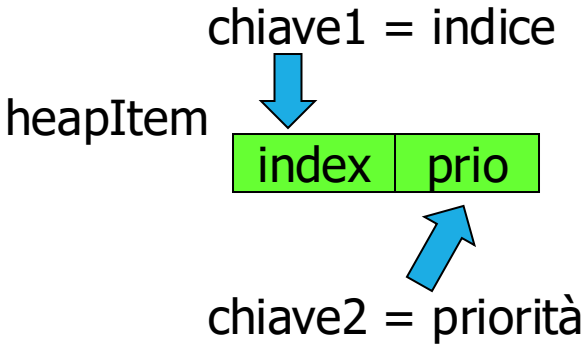
Soluzione #2

soluzione proposta

- la chiave dell'item deve essere univoca, senza possibili duplicati
- il modulo PQ usa la chiave dell'item per gestire una corrispondenza chiave-indice mediante una tabella di simboli efficiente (es. Hash table O(1), BST bilanciato O(lg(n))
- lo heap che realizza la coda a priorità contiene dati di un tipo heapItem interno alla PQ formato da coppie (indice, priorità). Si adotta la versione di «chiave affiancata al dato» (la priorità è un parametro aggiuntivo) invece che «chiave parte del dato»
- la PQ contiene Il vettore pq->qp (posizione in coda) serve per implementare una PQchange efficiente, identificando la posizione dell'elemento nello heap con costo O(1) (l'elemento è un indice) senza bisogno di una scansione lineare.




chiave = nome
item
name
prio


chiave1 = indice
heapItem
index
prio
chiave2 = priorità

L'ADT PQ è una struct con:

1. la tabella di hash pq->hash con dati di tipo Item gestisce la corrispondenza indice/dato e dato/indice con costo in media O(1)
2. la coda a priorità come heap implementato come vettore pq->A di dati di tipo heapItem
3. heapsize: intero per la dimensione corrente dello heap
4. il vettore pq->qp per memorizzare a quale indice nello heap si trova un dato di tipo heapItem

serve per implementare PQchange identificando la posizione dell'elemento nello heap.



maxN = 5

Utente

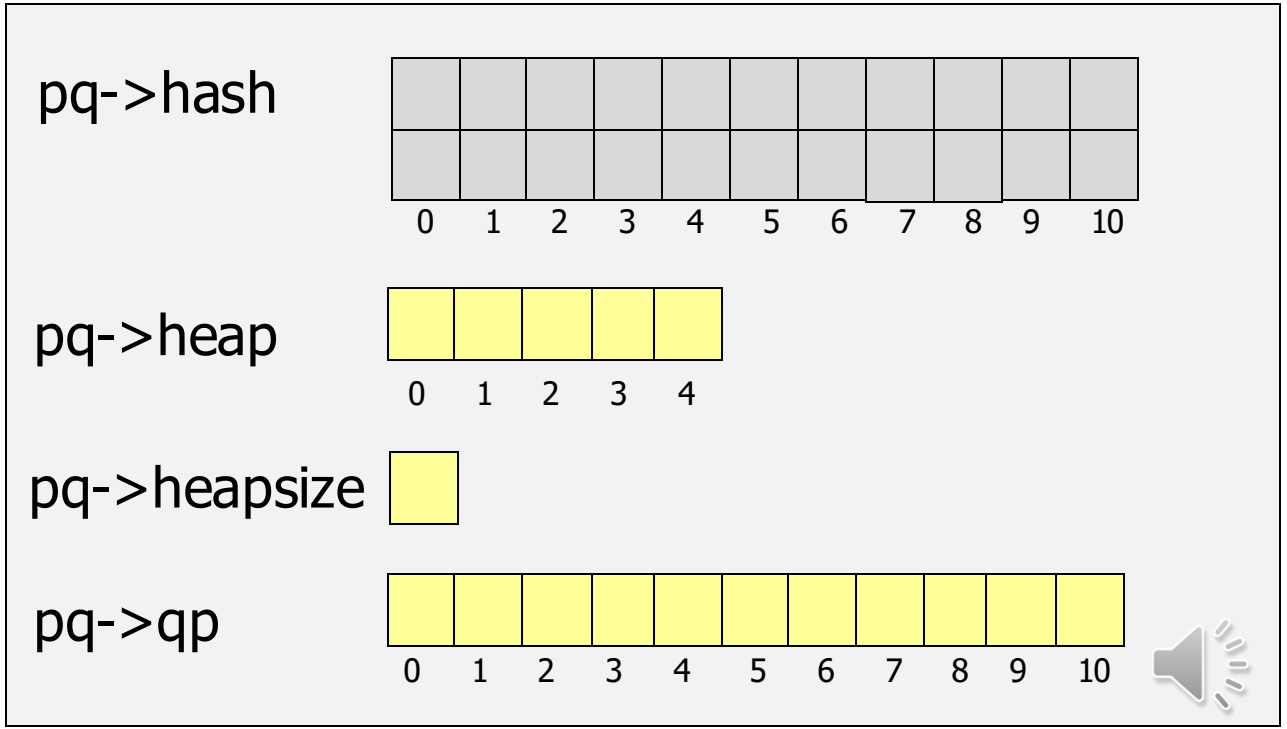
ADT I cat. coda a priorità di indici

item → PQinsert(pq, item)

item ← PQshowMax(pq)

item ← PQextractMax(pq)

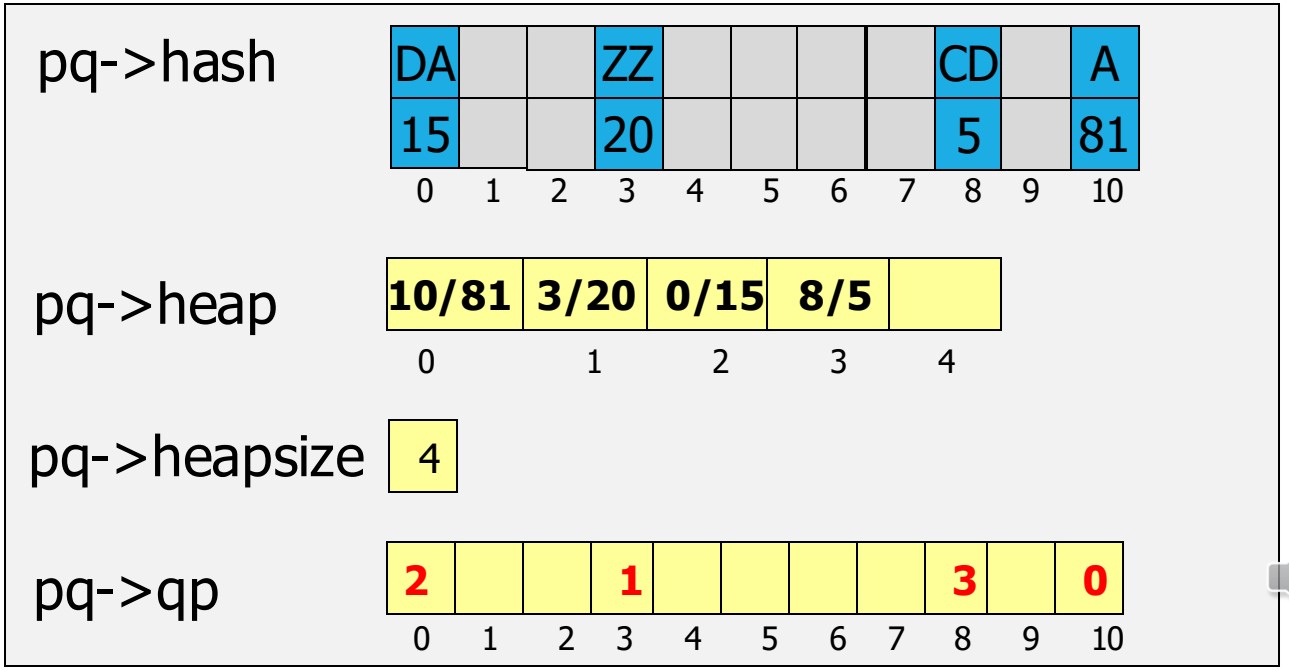
item → PQchange(pq, item)

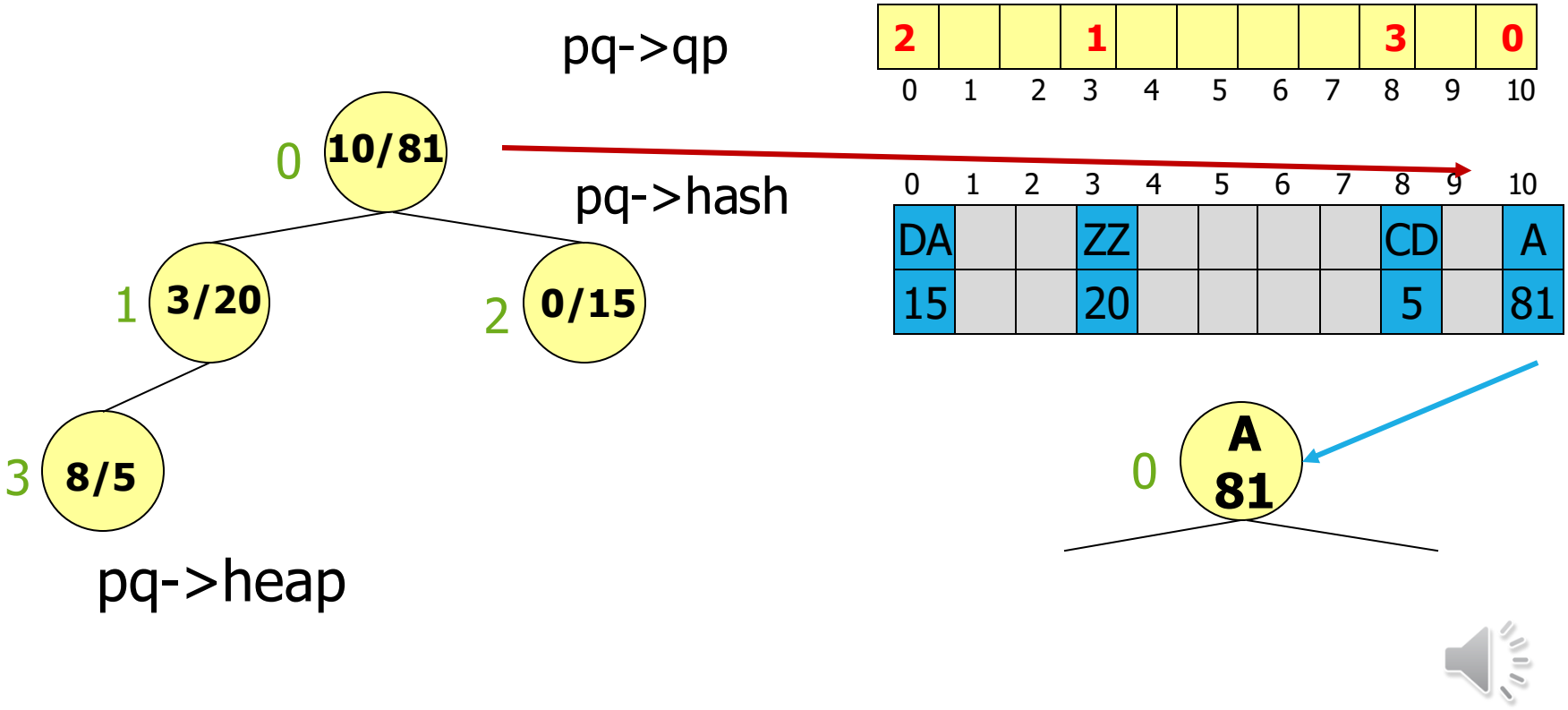


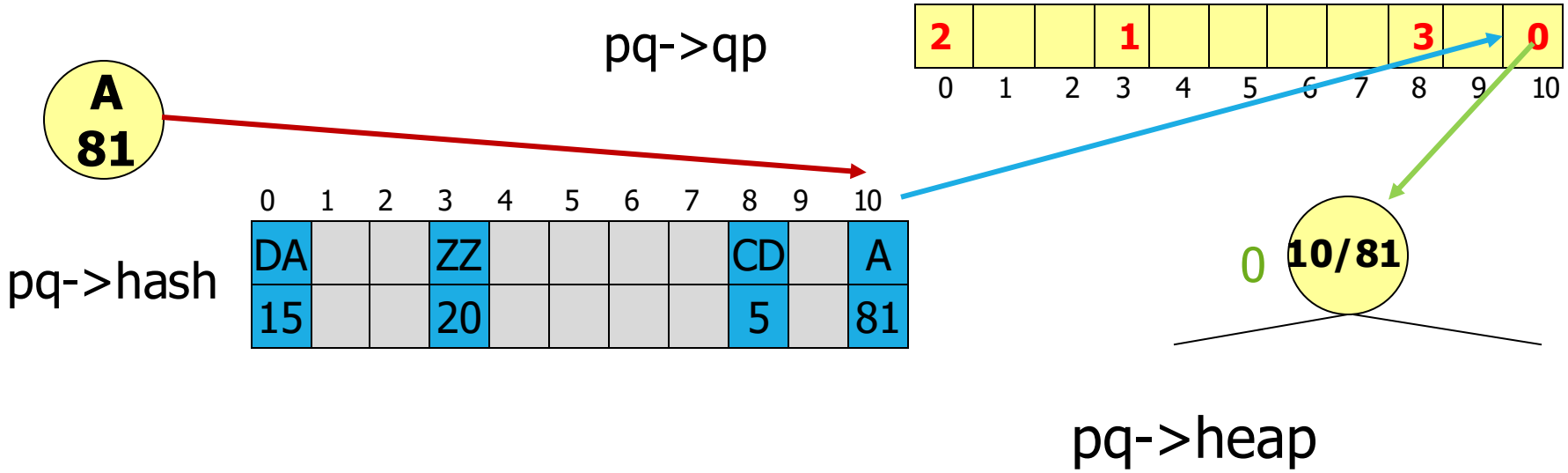
Esempio

Inserzione in sequenza in coda a priorità inizialmente vuota di:

- ZZ/20, A/81, DA/15, CD/5




pq->qp
2 1 3 0
0 1 2 3 4 5 6 7 8 9 10
pq->hash
0 1 2 3 4 5 6 7 8 9 10
DA ZZ CD A
15 20 5 81
0 10/81
1 3/20
2 0/15
3 8/5
pq->heap
0 A 81


A
81
pq->qp
2
1
3
0
0 1 2 3 4 5 6 7 8 9 10
0 1 2 3 4 5 6 7 8 9 10
DA ZZ CD A
15 20 5 81
pq->hash
0
10/81
pq->heap

PQ.h

```
typedef struct pqueue *PQ;

PQ      PQinit(int maxN);
void    PQfree(PQ pq);
int     PQempty(PQ pq);
int     PQsize(PQ pq);
void    PQinsert(PQ pq, Item val);
Item    PQshowMax(PQ pq);
Item    PQextractMax(PQ pq);
void    PQdisplay(PQ pq, int maxN);
void    PQchange(PQ pq, Item val);
```



PQ.c

```
...
typedef struct {int index; int prio;} heapItem;

struct pqueue {heapItem *heap; ST hash; int *qp; int heapsize; };

PQ PQinit(int maxN) {
  int i;
  PQ pq = malloc(sizeof(*pq));
  pq->heap = malloc(maxN*sizeof(heapItem));
  pq->hash = STinit(maxN);
  pq->qp = malloc(STsize(pq->hash)*sizeof(int));
  for (i=0; i < maxN; i++)
    pq->heap[i].index = -1;
  for (i=0; i < STsize(pq->hash); i++)
    pq->qp[i] = -1;
  pq->heapsize = 0;
  return pq;
}
```

Tipo di dato interno al modulo PQ per gestire la coppia (indice,priorità)

maxN è il massimo numero di dati nella coda a priorità



```
void PQfree(PQ pq) {
  free(pq->qp);
  STfree(pq->hash);
  free(pq->heap);
  free(pq);
}

int PQempty(PQ pq){
  return pq->heapsize == 0;
}

int PQsize(PQ pq) {
  return pq->heapsize;
}
```



Funzione PQinsert

Procede come nel caso di PQ di item:

- si inserisce l'elemento in pq->hash, ricavando il suo indice e la sua priorità
- si inseriscono indice e priorità in pq->heap, risalendo dal nodo corrente (inizialmente la foglia appena creata) fino al più alla radice in base al confronto tra priorità
- si mantiene aggiornato il vettore pq->qp.

Complessità: $T(n) = O(\lg n)$.



```
void PQinsert (PQ pq, Item item){
  int i, index, prio;
  index =STinsert(pq->hash, item);
  prio = STgetPrio(pq->hash, index);
  i = pq->heapsize++;

  while((i>=1) && (pq->heap[PARENT(i)].prio)<prio){
    pq->heap[i] = pq->heap[PARENT(i)];
    pq->qp[pq->heap[i].index] = i;
    i = PARENT(i);
  }
  pq->heap[i].index = index;
  pq->heap[i].prio = prio;
  pq->qp[index] = i;
}
```

inserisco il dato nella tabella di hash

risalgo nello heap cercando la posizione corretta

memorizzo il dato nella posizione corretta

aggiorno pq->qp



```
static void Swap(PQ pq, int pos1, int pos2){
  heapItem temp;
  int index1, index2;
  temp = pq->heap[pos1];
  pq->heap[pos1] = pq->heap[pos2];
  pq->heap[pos2] = temp;
  // update correspondence index-pos
  index1 = pq->heap[pos1].index;
  index2 = pq->heap[pos2].index;
  pq->qp[index1] = pos1;
  pq->qp[index2] = pos2;
}
```

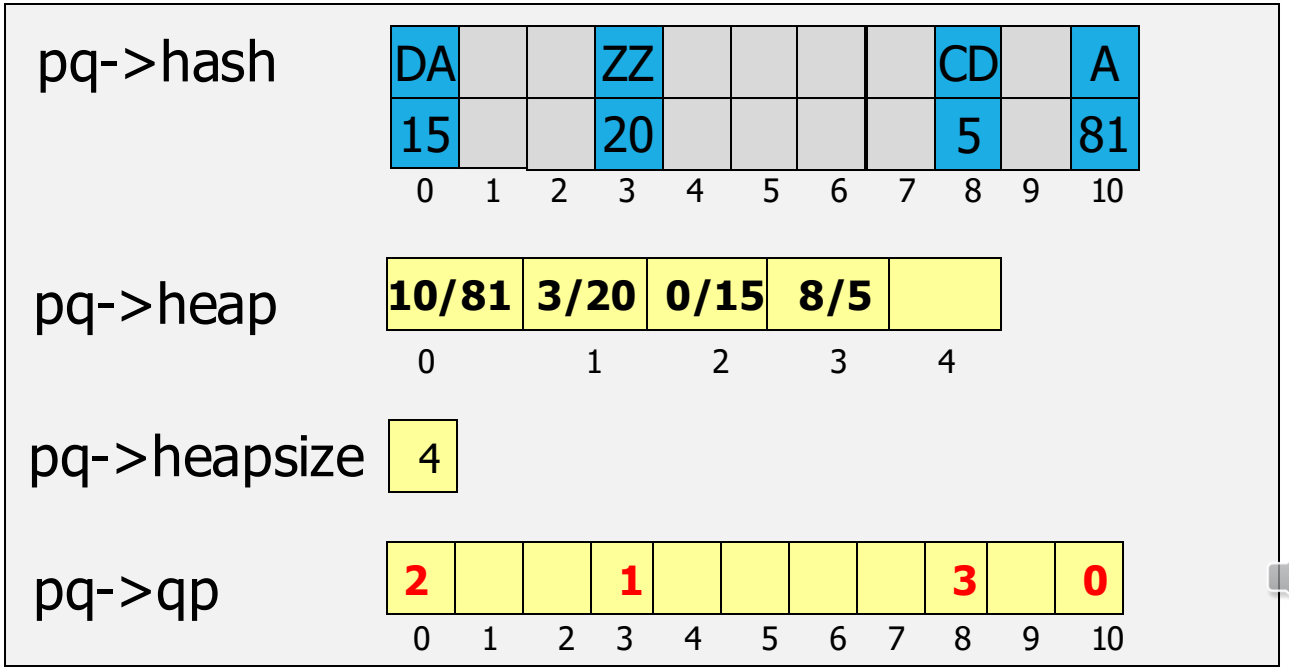


```
static void Heapify(PQ pq, int i) {
  int l, r, largest;
  l = LEFT(i);
  r = RIGHT(i);
  if (l<pq->heapsize && (pq->heap[l].prio>pq->heap[i].prio))
    largest = l;
  else
    largest = i;
  if (r<pq->heapsize && (pq->heap[r].prio>pq->heap[largest].prio))
    largest = r;
  if (largest != i) {
    Swap(pq, i, largest);
    Heapify(pq, largest);
  }
}
```



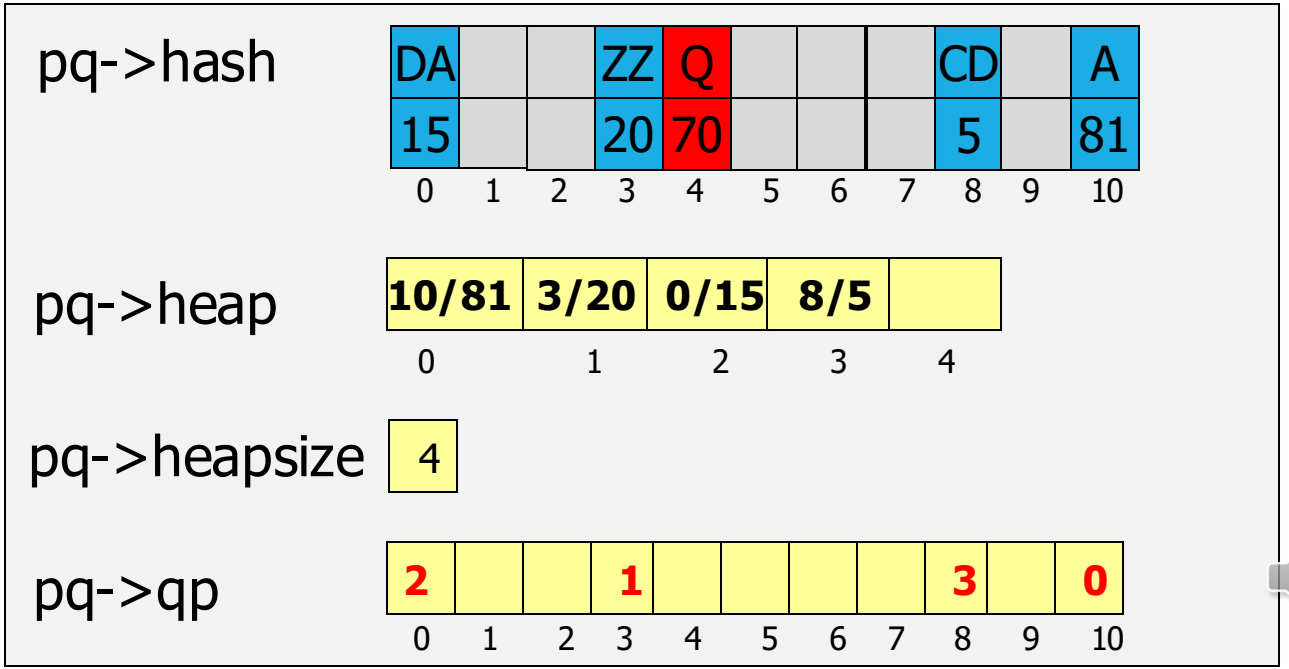
Inserzione di Q/70

pq



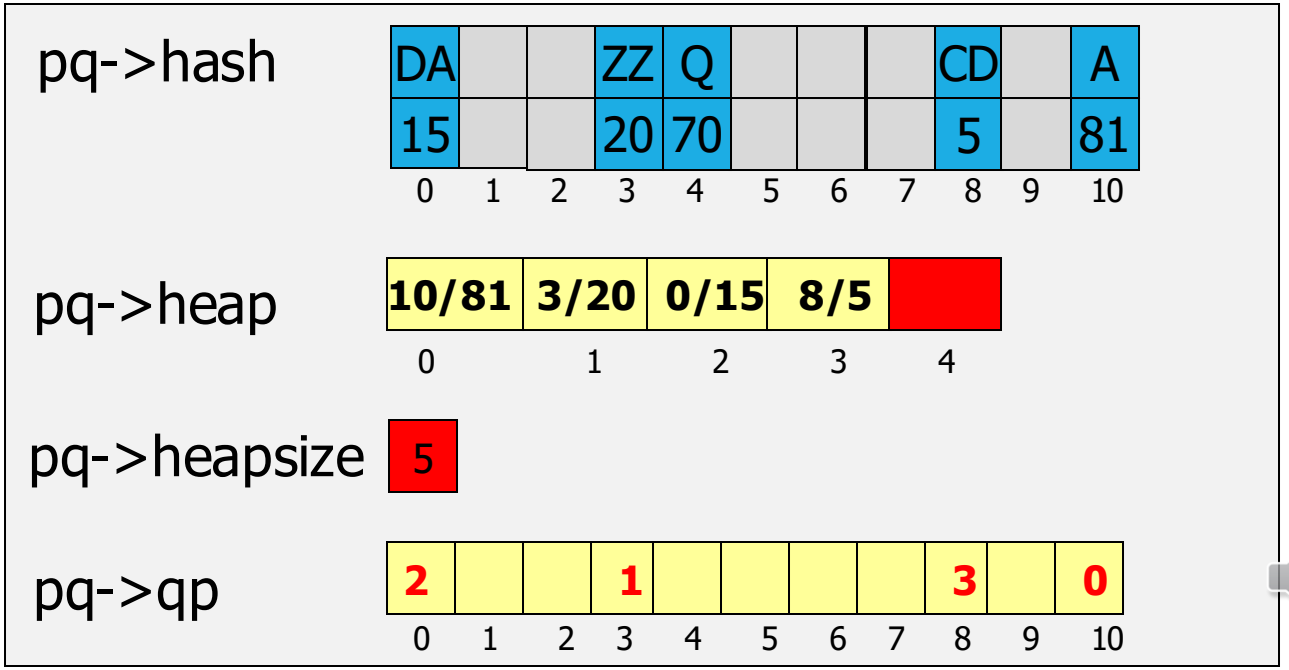
inserisco in pq->hash

pq



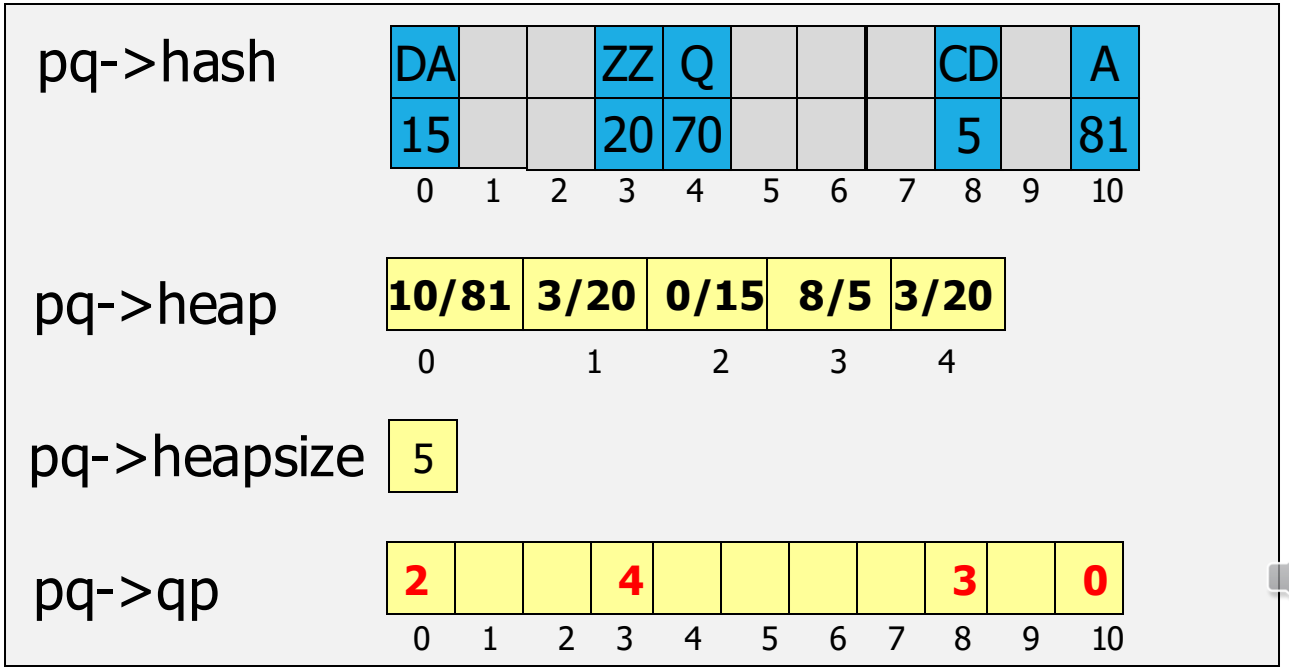
creo nuova foglia e aggiorno heapsize

pq



in pq->heap i =4 && 70 > 20, scende 3/20 (ZZ), si aggiorno pq->heap (all'indice i=4) e pq->qp (all'indice i=3)

pq



in pq->heap i =1 && 70 < 81, si memorizza 4/70 (Q), si aggiorna pq->qp (all'indice i=4)

pq

pq->hash

DA			ZZ	Q				CD		A
15			20	70				5		81
0	1	2	3	4	5	6	7	8	9	10

pq->heap

10/81	**4/70**	**0/15**	**8/5**	**3/20**
0	1	2	3	4

pq->heapsize

5

pq->qp

2			**4**	**1**				**3**		**0**
0	1	2	3	4	5	6	7	8	9	10

Funzione PQextractMax

- L’indice dell’item a massima priorità è nella radice dello heap
- tramite indice e tabella di hash si recupera l’item e lo si cancella dalla tabella di hash
- nello heap:
  - si scambia la radice con l'ultima delle foglie (quella più a destra nell'ultimo livello)
  - si riduce di 1 la dimensione dello heap
  - si ripristina le proprietà dello heap mediante applicazione di HEAPify
  - si aggiorna pq->qp.



```
Item PQextractMax(PQ pq) {
  int index;
  Item item;
  item = STgetItem(pq->hash, pq->heap[0].index);
  STdelete(pq->hash, item);
  Swap (pq, 0, pq->heapsize-1);
  index = pq->heap[pq->heapsize-1].index;
  pq->qp[index]=-1;
  pq->heapsize--;
  pq->heap[pq->heapsize].index=-1; // redundant
  Heapify(pq, 0);
  return item;
}
```



Funzione PQchange

- Tramite la tabella di hash pq->hash si ottiene l'indice dell'item e se ne cambia la priorità
- il vettore pq->qp, dato l'indice dell'item, ritorna con costo unitario la **posizione** dell'elemento nello heap
- si risale dalla posizione data fino al più alla radice confrontando la priorità del padre con la priorità modificata, facendo scendere l'heapItem del padre nel figlio se la priorità modificata è maggiore, altrimenti la si inserisce nel nodo corrente
- si applica HEAPify a partire dalla posizione data.



```
void PQchange (PQ pq, Item item) {
  int index, prio, pos;
  heapItem temp;
  index = STsearch(pq->hash, item);
  if (index == -1) { printf("Item not found!\n"); return; }
  else
    printf("found at index %d\n", index);
  prio = PRIOget(item);
  STchangePrio(pq->hash, index, prio);

  pos = pq->qp[index];
  temp = pq->heap[pos];
  temp.prio = prio; // new prio

  while ((pos>=1) && (pq->heap[PARENT(pos)].index < prio)) {
    pq->heap[pos] = pq->heap[PARENT(pos)];
    pq->qp[pq->heap[pos].index] = pos;
    pos = PARENT(pos);
  }
  pq->heap[pos] = temp;
  pq->qp[index] = pos;
  Heapify(pq, pos);
}
```

indice in pq->hash

cambio priorità

posizione in pq->heap

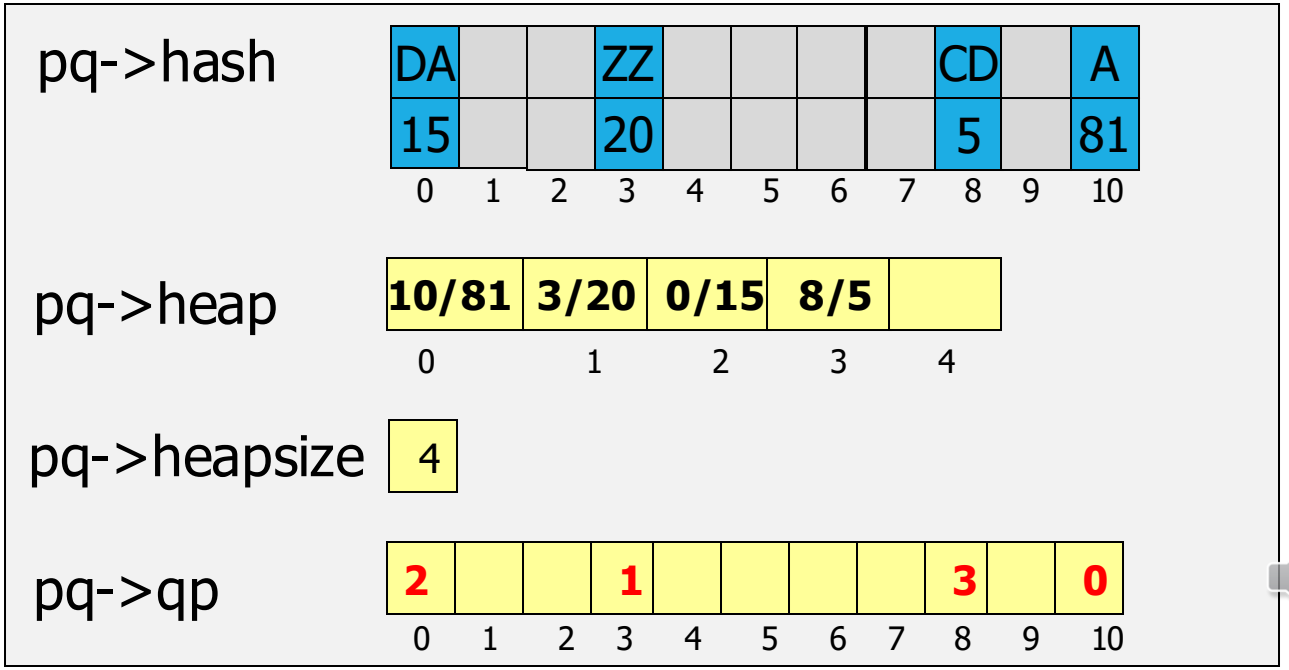
posizionamento corretto in risalita

aggiornamento di pq->heap e pq->qp

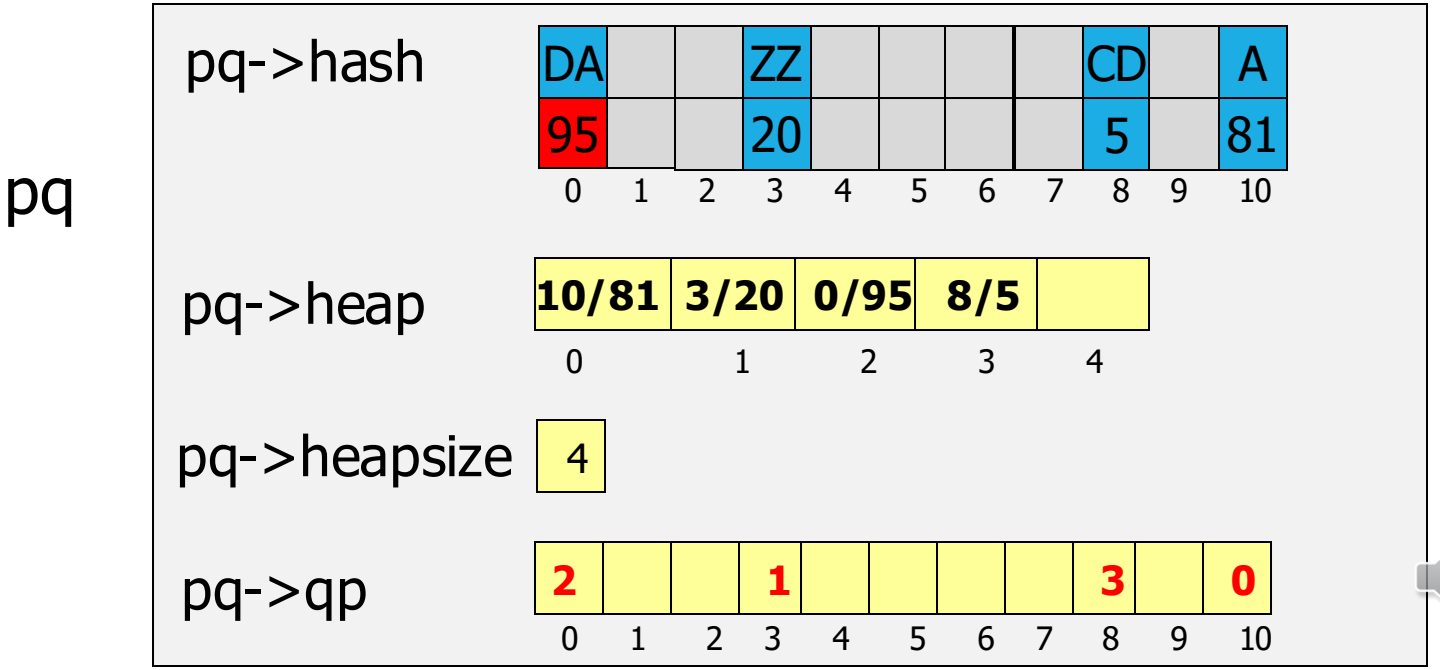
posizionamento corretto in discesa

Esempio: DA da 15 a 95

pq

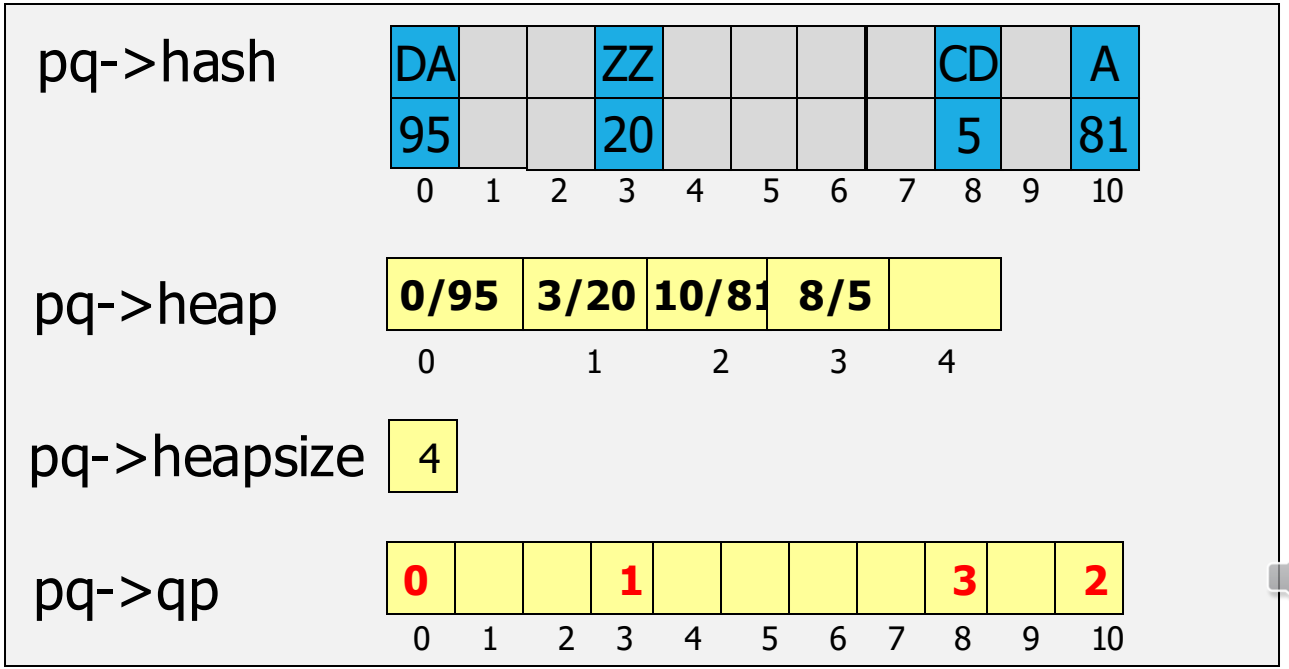


da pq->hash si ricava l'indice di DA (0) e si aggiorna la priorità
da pq->qp si ricava l'indice di DA (2) in pq->heap e si aggiorna la priorità



si procede con fixup

pq

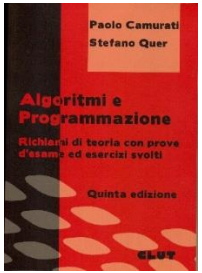


Riferimenti

- Heap:
  - Cormen 7.2, 7.3
  - Sedgewick 9.2, 9.3
- Heapsort:
  - Cormen 7.4
  - Sedgewick 9.4
- Code a priorità:
  - Cormen 7.5
  - Sedgewick 9.1, 9.6



Esercizi di teoria



Code a priorità e heap

- Heap
- Heap Sort
- Code a priorità di dati